

**MedIDS: A lightweight hybrid Machine Learning Intrusion Detection System (IDS)
for Internet of Medical Things (IoMT) environments Using Network Traffic Analysis
and Malware Image Classification on Raspberry Pi**

Alsafa Abdullah Hilal Al-Wahshi

A thesis submitted in partial fulfilment of the requirements for the

Degree of Bachelor of Science
in
Cybersecurity

Department of Computer Science

College of Engineering and Computer Sciences

German University of Technology in Oman

Sultanate of Oman

5th of June 2026



MedIDS: A lightweight hybrid Machine Learning Intrusion Detection System (IDS) for Internet of Medical Things (IoMT) environments Using Network Traffic Analysis and Malware Image Classification on Raspberry Pi

Alsafa Abdullah Hilal Al-Wahshi

A thesis submitted in partial fulfilment of the requirements for the

Degree of Bachelor of Science
in
Cybersecurity

Department of Computer Science

College of Engineering and Computer Sciences

German University of Technology in Oman

Sultanate of Oman

5th of June 2026

©

Student's Name Alsafa Abdullah Hilal Al-Wahshi

(I.D. #) 21-0038

Title of Thesis: MedIDS: A lightweight hybrid Machine Learning Intrusion Detection System (IDS) for Internet of Medical Things (IoMT) environments Using Network Traffic Analysis and Malware Image Classification on Raspberry Pi

Thesis Committee:

- 1. Supervisor** Dr.Waseem Raja Anwar
 Title Assistant Professor
 Department Computer Science
 Institution German University of Technology in Oman

- 2. Co-Supervisor** Dr.Abdelaziz Amara Kobra
 Title Assistant Professor
 Department Computer Science
 Institution German University of Technology in Oman

Declaration

I hereby declare that this thesis is my own work and effort and that it has not been submitted anywhere for any award. Where other sources of information have been used, they have been acknowledged.

A handwritten signature in black ink, appearing to be a stylized 'K' or similar character.

Date: 5th of June 2026

Signature:

Dedication

To all patients and healthcare professionals, whose safety, privacy, and trust form the foundation of healthcare. I hope that this work contributes, even in a small way, to enhancing the security and resilience of modern healthcare systems.

Abstract

The Internet of Medical Things (IoMT) has emerged as an important part of modern healthcare as it allows real-time monitoring of patients and improves the delivery of healthcare services via connected medical devices. This development has exposed the network environment of healthcare facilities to several cyber threats, including unauthorized access, data thefts, ransomware, and even denial-of-service. Furthermore, the resource-constrained nature of many IoMT devices makes the deployment of traditional security mechanisms challenging.

This research introduces MedIDS, a lightweight hybrid Machine Learning-based Intrusion Detection System designed to improve the security of IoMT environments. The framework combines network-based intrusion detection and image-based malware detection implemented in a single architecture suitable for deployment on lightweight edge devices. For network intrusion detection, various ML-based classifiers have been tested using CICIoMT2024 dataset including Decision Tree, Logistic Regression, and Naïve Bayes algorithms. For malware image detection, a lightweight MobileNetV2 deep learning algorithm was trained using the MaleX dataset. All these ML algorithms were implemented on a Streamlit-based real-time monitoring platform running on a Raspberry Pi Zero 2W edge device.

The experimental results revealed that the best performance of network intrusion detection was delivered by the Decision Tree algorithm, where the accuracy rate was 99.21%, precision rate was 75.35%, recall rate was 98.87%, and F1-Score was 85.52%. The MobileNetV2 model achieved 76.33% accuracy, 100% precision, 52.67% recall, and 69.00% F1-score for malware image classification. Lightweight deployment results showed that the network-based IDS required around 216 MB of memory when operating normally and 321 MB during attacks with a detection latency of 0.00123 seconds for benign data packets and 0.03971 seconds during attacks. The image-based IDS used approximately 164–166 MB of memory with inference latencies of 0.08307 seconds for benign images and 0.08196 seconds for malware images.

The findings indicate that MedIDS effectively delivers real-time intrusion detection, malware image classification, and alert generation, while maintaining operational efficiency on resource-constrained edge devices. This framework offers a lightweight hybrid security solution that supports secure and scalable Internet of Medical Things (IoMT) healthcare environments.

Keywords: Internet of Medical Things (IoMT), Unauthorized access, Ransomware, Denial-of-services, Resource-constrained, MedIDS, Intrusion Detection System (IDS), Machine Learning, Deep Learning, Decision Tree, MobileNetV2, Raspberry Pi.

Table of contents

Chapter 1: Introduction	1
1.1 Statement of the Problem.....	2
1.2 Purpose of the Research.....	2
1.3 Research Objectives.....	3
1.4 Research Questions.....	3
1.5 Research Outline	4
Chapter 2: Literature Review.....	5
2.1 Intrusion Detection Systems for IoMT	6
2.2 Machine Learning and Deep Learning-Based Intrusion Detection Systems for the Internet of Medical Things.....	7
2.3 Lightweight and hybrid Edge-based IDS Solutions.....	8
2.4 Research gaps and challenges	11
3. Research Methodology	12
3.1 System Architecture	12
3.2 Data Description and Preprocessing	13
3.2.1 Description and Preprocessing Methods of CICIoMT2024 Dataset	14
3.2.2 Description and Preprocessing Methods of MaleX Dataset	16
3.3 Lightweight Machine Learning and Deep Learning Development	18
3.3.1 Network-Based ML Models.....	19
3.3.2 Image-Based DL Models	20
3.4 Validation Approach and Experimental Setup	21
3.4.1 Performance Evaluation Metrics.....	21
3.4.2 Edge Device Performance Measurements	22
3.4.3 Experimental Environment	22
4. Implementation	24
4.1 Network-Based IDS Implementation.....	24
4.1.1 CICIoMT2024 Dataset Processing Implementation.....	24
4.1.2 Machine Learning Models Training.....	28
4.1.3 Real-time Network Traffic Monitoring and Detection	29
4.2 Image-Based IDS Implementation.....	31

4.2.1 MaleX Dataset Processing Implementation.....	31
4.2.2 MobileNetV2 Model Training	35
4.2.3 Real-time Malware Image Detection	37
4.3 Raspberry Pi Edge Deployment and MedIDS Interface	39
4.3.1 Raspberry Pi Edge Deployment Environment.....	39
4.3.2 MedIDS Interface.....	42
5. Results and Discussion	44
5.1 Performance of Network-based Intrusion Detection System.....	44
5.1.1 Machine Learning Models Evaluation.....	44
5.1.2 Raspberry Pi Edge Deployment Performance of the Network-based IDS	54
5.2 Performance of Image-based Intrusion Detection System.....	57
5.2.1 MobileNetV2 Model Evaluation.....	57
5.1.2 Raspberry Pi Edge Deployment Performance of the Image-based IDS	59
5.3 Real-Time Attack Simulation and Detection Results	61
5.3.1 Real-Time Network Attack Detection.....	61
5.3.2 Real-Time Malware Image Detection.....	63
Chapter 6. Conclusion.....	66
6.1 Conclusion	66
6.2 Limitations	67
6.3 Future Works.....	68
Appendix.....	69
References.....	70

List of tables

Table 1. Summary of IDS approaches discussed in the literature review.	10
Table 2. Raspberry Pi Zero 2W deployment environment.....	23
Table 3. Performance comparison of network-based IDS machine learning models.	53
Table 4. Raspberry Pi Zero 2W resource utilization of the network-based IDS.....	57
Table 5. Raspberry Pi Zero 2W resource utilization of the image-based IDS.	60

List of figures

Figure 1. Architecture of an IoMT environment.....	5
Figure 2. Overall architecture of the proposed MedIDS framework	13
Figure 3. CICIoMT2024 Dataset Preprocessing Workflow.....	16
Figure 4. MaleX Dataset Preprocessing Workflow	18
Figure 5. CSV merging, traffic labelling, and dataset sampling	25
Figure 6. Class distribution before under-sampling.....	25
Figure 7. Class distribution after under-sampling.....	26
Figure 8. Under-sampling implementation	26
Figure 9. Implementation of label encoding	26
Figure 10. Feature-label separation and feature alignment implementation.....	27
Figure 11. Implementation of feature selection and feature normalization	27
Figure 12. Training implementation of Decision Tree, Naïve Bayes, and Logistic Regression models.	28
Figure 13. Evaluation function	29
Figure 14. Importing libraries and loading the trained DT model	29
Figure 15. NFStream real-time traffic monitoring.....	30
Figure 16. Feature mapping function.....	30
Figure 17. Malware and benign image folder loading and random sampling	32
Figure 18. Visualization of malware and benign image samples using Matplotlib.	33
Figure 19. Image preprocessing parameters and labeling technique initialization.	34
Figure 20. Dataset splitting	34
Figure 21. TensorFlow dataset pipeline construction	35
Figure 22. MobileNetV2 model architecture with a custom classification layer	36
Figure 23. Model compilation and metrics initialization.....	36
Figure 24. MobileNetV2 training using TensorFlow model.fit() and callback mechanisms....	37
Figure 25. Importing libraries for image-based IDS and loading the TensorLite model.....	38
Figure 26. Image preprocessing and prediction	38
Figure 27. Raspberry Pi OS installation and device configuration.....	40
Figure 28. Raspberry Pi SSH remote access.....	40
Figure 29. Raspberry Pi operating system and Python environment verification	41
Figure 30. Successful execution of network-based IDS on Raspberry Pi	41
Figure 31. Successful execution of image-based IDS on Raspberry Pi.....	41
Figure 32. Streamlit-based MedIDS application running on Raspberry Pi	42
Figure 33. Network-based IDS monitoring interface	43
Figure 34. Image-based malware detection interface	43
Figure 35. Performance evaluation of the Decision Tree model using original features .45	
Figure 36. Confusion matrix of the Decision Tree model using original features.....	45
Figure 37. Performance evaluation of the Decision Tree model using feature selection.	46
Figure 38. Confusion matrix of the Decision Tree model using feature selection.	46
Figure 39. Performance evaluation of the Decision Tree model using feature scaling.	46
Figure 40. Confusion matrix of the Decision Tree model using feature scaling.	47

Figure 41. Performance evaluation of the Naïve Bayes model using original features.....	47
Figure 42. Confusion matrix of the Naïve Bayes model using original features.....	48
Figure 43. Performance evaluation of the Naïve Bayes model using feature selection.	48
Figure 44. Confusion matrix of the Naïve Bayes model using feature selection	49
Figure 45. Performance evaluation of the Naïve Bayes model using feature scaling.	49
Figure 46. Confusion matrix of the Naïve Bayes model using feature scaling.	50
Figure 47. Performance evaluation of the Logistic regression model using original features.	50
Figure 48. Confusion matrix of the Logistic Regression model using original features.	51
Figure 49. Performance evaluation of the Logistic regression model using feature selection.	51
Figure 50. Confusion matrix of the Logistic Regression model using feature selection.....	52
Figure 51. Performance evaluation of the Logistic regression model using feature scaling. ..	52
Figure 52. Confusion matrix of the Logistic Regression model using feature scaling.....	52
Figure 53. Comparison of accuracy, precision, recall, and F1 score among evaluated network-based IDS machine learning models.....	54
Figure 54. Resource utilization of Raspberry Pi Zero 2W before IDS monitoring.	55
Figure 55. Resource utilization during benign traffic monitoring.	56
Figure 56. Resource utilization during attack traffic monitoring.....	56
Figure 57. Performance evaluation of the MobileNetV2 model.....	58
Figure 58. Confusion matrix of the MobileNetV2 malware image classifier.....	58
Figure 59. Resource utilization during benign image classification on Raspberry Pi Zero 2W.	59
Figure 60. Resource utilization during malware image classification on Raspberry Pi Zero 2W.	60
Figure 61. Raspberry Pi network configuration and target IP identification.	61
Figure 62. Real-time benign traffic monitoring using MedIDS.....	62
Figure 63. Real-time UDP flood attack detection and alert pop-up.....	62
Figure 64. Email notification after network attack detection.	63
Figure 65. Benign image classification.....	64
Figure 66. Malware image classification.	65
Figure 67. Email notification after malware image detection.....	65

Chapter 1: Introduction

While the world moves towards advanced technologies, the main objective of developing an excellent healthcare system is to leverage robust and secure medical technologies for high-quality preventive and clinical care. Therefore, many hospitals have started integrating Internet of Medical Things (IoMT) devices into their medical systems. IoMT refers to any medical device connected to the internet that can collect and measure at least one patient's health data, such as heart rate, body temperature, and blood pressure [1]. As a result, the use of IoMT devices has significantly improved patients' vital signs and medical conditions [2].

However, despite the benefits of IoMT systems to healthcare, the increasing number of connected medical devices has also expanded the healthcare attack surface. Cybercriminals are increasingly targeting medical systems because IoMT and patient data are considered nearly 50 times more valuable than data from other sectors [3]. Research shows that approximately 99% of healthcare institutions have at least one medical device with known vulnerabilities, putting their network at risk. In addition, IoMT are resource-constrained nature with limited processing resources, power, and limited security considerations, making it more vulnerable to cyberattacks such as ransomware, unauthorized access, data breaches, and distributed denial-of-service (DDoS) attacks [4].

Intrusion Detection Systems (IDS) are essential for addressing cybersecurity challenges in healthcare networks. IDS monitors network traffic and system activities to detect malicious behavior, unauthorized access, and cyberattacks. Unlike traditional preventive measures, IDS provides an additional layer of protection by identifying suspicious activities and alerting security administrators. In Internet of Medical Things (IoMT) environments, IDS is especially important because many medical devices have limited computational resources and cannot support complex security solutions. As a result, lightweight IDS approaches are increasingly used to secure IoMT devices while maintaining system efficiency [5].

This paper presents MedIDS, a lightweight hybrid Machine Learning-based Intrusion Detection System (ML-IDS) developed to strengthen the security, integrity, and confidentiality of IoMT devices. The proposed system combines both image-based and network-based intrusion detection approaches using two supervised ML models, namely, Decision Tree and

MobileNetV2. Therefore, it can detect intrusions from both network traffic and malware image classification. Furthermore, the proposed system is designed for deployment in edge-computing environments using low-resource devices such as Raspberry PI, which are widely used in IoMT devices because of their low cost and low power consumption.

The main objective of this paper is to create a lightweight hybrid IDS that can detect attacks in IoMT environments using both real-time Network traffic analysis and malware image classification, while generating real-time security alerts. The proposed framework enables efficient edge deployment, enhances intrusion detection capabilities, and improves security within resource-constrained healthcare environments.

1.1 Statement of the Problem

The use of IoMT devices within healthcare infrastructure has increased significantly in recent years. However, these systems continue to face security challenges. Due to IoMT's limited computing resources, such as CPU power and memory, many IoMT devices and lightweight edge devices, such as Raspberry Pi, cannot efficiently support traditional intrusion detection systems (IDS), which are often slow, expensive, and resource-intensive. Moreover, most existing IDS solutions focus on network traffic analysis, while ignoring attacks coming from other sources, such as malware images.

1.2 Purpose of the Research

The main goal of this research is to investigate the effectiveness of lightweight Machine Learning (ML) and Deep Learning (DL) models for IDS in the IoMT devices. The research aims to provide a lightweight solution to protect medical systems from cyber threats and while remaining suitable for deployment on resource-constrained IoMT and edge devices. In addition, this research will focus on evaluating different machine learning and deep learning models through analyzing their accuracy, precision, recall, F1 score, and overall detection performance. The goal is to build a lightweight hybrid system that combines both image-based and network-based intrusion detection for detecting attack traffic and malware image samples, supporting edge-device deployment and real-time alert generation.

1.3 Research Objectives

The main objective of this research is to create a lightweight hybrid Machine Learning Intrusion Detection System for securing IoMT devices. The main objectives are:

- To preprocess and prepare two datasets, namely, CICIoMT2024, containing network traffic samples, and MaleX with malware image classification. Including dataset cleaning and balancing, encoding, feature selection, normalization, and image preprocessing techniques.
- To train various lightweight ML and DL models for both network-based and image-based intrusion detection.
- To evaluate the model performance using accuracy, precision, recall, and F1 score.
- To identify the best and most effective lightweight model for IoMT intrusion detection.
- To design a lightweight hybrid IDS combining both network flow analysis and malware image classification.
- To deploy the framework on a Raspberry Pi edge device and support real-time intrusion detection and alert generation.
- To evaluate the resource efficiency of MedIDS on a Raspberry Pi Zero 2W edge device using CPU utilization, memory consumption, and detection latency measurements.

1.4 Research Questions

To achieve the stated objectives, the following questions will be addressed:

- How can the CICIoMT2024 and MaleX datasets for hybrid IDS in IoMT environments be successfully pre-processed and prepared?
- How effective are different Machine Learning and Deep Learning models in detecting intrusions and malware in terms of accuracy, precision, recall, and F1 score?

- Which lightweight model provides the best performance and efficiency for network-based intrusion detection in IoMT?
- Can lightweight edge devices, such as Raspberry Pi, support real-time intrusion detection and alert generation in IoMT environments?

1.5 Research Outline

This paper is organized into six chapters. Chapter 1 is the introduction chapter, which highlights the importance of cybersecurity in IoMT environments and discusses the research problem, objectives, scope, and questions. Chapter 2 is the literature review, which examines traditional IDS, different ML and DL approaches, and existing lightweight IDS. Chapter 3 presents the research methodology, which explains the MedIDS solution, its architecture, datasets, data preprocessing techniques, different ML and DL methods used, and evaluation procedures. Chapter 4 describes the implementation of MedIDS, which presents both image-based and network-based intrusion detection pipelines, including model training, evaluation, real-time alert generation, and lightweight edge device deployment using Raspberry Pi. Chapter 5 presents the experimental results, including the evaluation of detection performance, latency, CPU utilization, memory consumption, real-time attack detection, and deployment efficiency. Finally, Chapter 6, the conclusion and future work chapter, highlights research findings, discusses the study's limitations and goals' accomplishments, and makes recommendations for future improvements.

Chapter 2: Literature Review

IoMT has successfully brought an enormous technological transformation to the modern healthcare system. Figure 1 illustrates an IoMT network, in which patients' vital signs are collected using medical devices such as smart wearable devices and Electronic Medical Records (EMR). The collected data is then sent to the IoMT application through the internet, allowing information to reach healthcare professionals and sending a response back to patients [6]. However, IoMT still faces significant security challenges. Intrusion Detection mechanisms can help ensure security in IoMT Networks. Much research exists in this field, but it still treats network-based detection, image-based malware detection, and lightweight edge deployment as separate problems. This literature review discusses multiple research papers related to IDS and IoMT security and highlights the challenges, limitations, and research gaps targeted by the proposed MedIDS system.

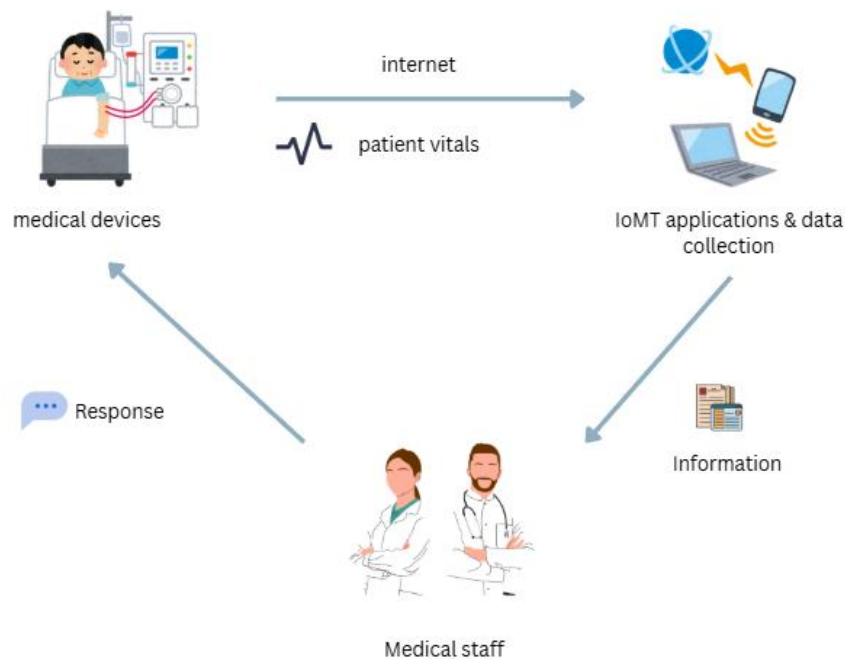


Figure 1. Architecture of an IoMT environment.

2.1 Intrusion Detection Systems for IoMT

An Intrusion Detection System (IDS) is a security tool that analyzes network activity and detects abnormal or malicious behavior [7]. These systems help to defend IoMT networks against different security threats that potentially compromise medical devices and patients' data. The IDS is especially important in IoMT networks due to the heterogeneity of devices, limited resources, and exposure to complex traffic patterns, which traditional security tools often cannot address effectively [5].

In [5], the author explains that conventional IDS methods are grouped into several approaches. The simplest type of IDS is signature-based, which searches for known attack signatures. This makes it effective against known attacks and reduces the false positive rate. However, it lacks the capability of detecting zero-day attacks. Another approach is anomaly-based IDS, which identifies deviations from normal behavior patterns. It gives better insight into unknown attacks, but this approach usually results in more false alarms and reduces efficiency in IoMT environments. Both methods have advantages and limitations that make them less appropriate for modern IoMT environments where attack patterns are continuously changing.

From an architecture perspective, paper [5] states that traditional network-based IDS is more practical in IoMT environments than host-based systems because of their limited memory and computing power. It monitors traffic between devices at the gateway or network level, rather than installing resource-intensive software on each device. However, many traditional network-based IDS rely on centralized monitoring, which can result in scalability, latency, and computing challenges. In healthcare settings, these problems are even more important because late intrusion detection can impact the availability and reliability of medical services.

An example of a traditional lightweight solution is the Snort-based edge gateway on Raspberry Pi mentioned in [8]. In this study, Snort was deployed as a lightweight traditional IDS on low-cost hardware, detecting only 60% of attacks. This evidence demonstrates that Snort-based IDS can operate effectively on low-cost devices, but also faces some limitations, such as failing to detect attacks that do not match predefined rules.

These studies show that the main limitation of conventional IDS includes not only detection accuracy but also restricting adaptability, along with high deployment overhead in IoMT environments. Therefore, Intelligent Machine Learning and Deep Learning lightweight

IDS approaches have become necessary, as they offer better detection abilities with reduced computational demands.

2.2 Machine Learning and Deep Learning-Based Intrusion Detection Systems for the Internet of Medical Things

Researchers are now using ML and DL intrusion detection approaches to achieve better detection of known and unseen attacks, rather than older traditional detection methods. According to research [5], ML and DL techniques have become essential for medical IDS research due to their ability to learn and adapt. Various models, such as decision trees, random forests, support vector machines (SVM), k-nearest neighbors (KNN), long short-term memory (LSTM), and convolutional neural networks (CNN), are being trained and evaluated.

Several studies discussed the effectiveness of ML and DL models for IoMT intrusion detection. Recent work developed a hybrid framework that uses Machine Learning models with feature selection techniques. It trained Random Forest, XGBoost, KNN, and Logistic Regression models on the CICIoMT2024 dataset, achieving an accuracy of 99.81%[9]. Another study proposed an explainable XAI-XGBoost solution with 99.22% accuracy on the WUSTL-EHMS-2020 dataset. It shows that tree-based models can be accurate and interpretable in healthcare [10]. Furthermore, a stacking ensemble DL-based solution achieved 99.1% accuracy on the ECU-IoHT dataset. The ensemble approach may present computing challenges for resource-limited devices [11]. These results indicate that ML and DL techniques can achieve high performance in IoMT environments.

For malware image classification, MobileNetV2, with its light architecture, has become a prominent model. Paper [12] introduced a visualized malware multi-classification framework using a fine-tuned convolutional neural network (CNN). The binary files were converted to 2D images and then forwarded to 8 different models for detection, including MobileNet-V2. The system accomplished 99.97% accuracy on the Maling dataset. Research [13] discusses another application of the MobileNetV2 model, which uses network attack classification by transforming tabular traffic data into grayscale images, achieving 97% accuracy in the UNSW-NB15 dataset. These findings confirm the ability of MobileNetV2 for lightweight edge-based detection.

Most machine learning (ML) and deep learning (DL) based intrusion detection system (IDS) solutions have limitations. They focus on either network traffic analysis or malware image classification. Most existing models achieve high accuracy but require computing resources beyond the capabilities of lightweight IoMT devices [11]. This demonstrates the need to build a lightweight, hybrid, edge-deployable IDS solution that integrates multiple detection sources and can work on IoMT devices.

2.3 Lightweight and hybrid Edge-based IDS Solutions

IoMT devices are resource-constrained, requiring a lightweight IDS with low computational cost, memory usage, and energy consumption, designed to run on low-power devices and maintain acceptable detection performance while enabling fast decision-making and scalability [5].

Edge computing enables lightweight IDS in IoMT, processing data locally on gateway or edge devices instead of sending it to a centralized cloud server. This approach reduces latency, preserves bandwidth, and improves response times for critical healthcare applications. Consequently, edge-based intrusion detection system (IDS) architectures are increasingly adopted in Internet of Medical Things (IoMT) security applications.

Recent advancements in lightweight edge intrusion detection system (IDS) solutions demonstrate the feasibility of deploying intrusion detection on devices with constrained resources. The RealGuard solution introduced in [14] is a lightweight DNN-based Network IDS that runs on IoT gateways such as Raspberry Pi and achieves an accuracy of 99.57%, indicating the ability of Raspberry Pi devices to support lightweight intrusion detection tasks while maintaining acceptable performance. Another study introduces an intelligent SaaS-based IDS for resource-constrained IoMT, using SHAP for interpretability, PSO for feature engineering, and ensemble models deployed at the edge, achieving an accuracy of 96.57% [15].

Moreover, recent studies have also explored hybrid IDS approaches. IntruSafe is another system built using two models: a Fully Connected Convolutional Neural Network (FCNN) and a Long Short-Term Memory (LSTM) trained with the CICIoT2023 dataset as the network dataset and Maling as the image dataset, and achieved an accuracy of approximately 97% for both. This system has combined both network traffic analysis and malware image classification and deployed on a platform rather than a lightweight edge device, indicating a gap in edge-deployable hybrid IDS solutions for IoMT [16].

Most existing lightweight edge-deployment IDS solutions focus on network traffic analysis, while ignoring malware image detection. This is because of the limited computational capabilities of edge devices, restricting IoMT IDS from analyzing image patterns. The need to integrate a hybrid IDS for network and image detection has become a big challenge in lightweight edge devices.

Table 1 summarizes the existing IDS studies discussed in this chapter, grouped by their detection category to highlight how each area supports the case of MedIDS. Previous studies show strong results in network traffic analysis, malware image classification, or lightweight edge deployment. However, none of the reviewed approaches combine network-based intrusion detection, malware image classification, edge deployment on Raspberry Pi, and real-time alert generation in a single framework.

Table 1. Summary of IDS approaches discussed in the literature review.

Category	References	Method	Dataset/traffic source	Accuracy (%)	Edge deploymet?	image detection?	Key limitation
Traditional IDS	[8]	Snort	General traffic	~ 60.00	Yes	No	Unable to detect unknown attacks
ML /DL Network-based IDS	[9-11]	- Random Forest, - XGBoost, KNN, - Logistic Regression - XAI-XGBoost - Ensemble DL	- CICIoMT2024 - WUSTL- - EHMS-2020 - ECU-IoHT	- 99.81 - 99.22 - 99.10	No	No	Present computing challenges for resource-limited devices, and focus on network traffic only
ML/DL image-based IDS	[12,13]	CNN, MobileNetV2	- Malimg - UNSW-NB15	- 99.97 - 97.00	Partial	Yes	Absence of network traffic analysis capabilities
Lightweight Edge IDS	[14,15]	- DNN - Ensemble models	IoMT network traffic	- 99.75 - 96.57	Yes	No	Focused on network intrusion detection
Hybrid IDS	[16]	FCNN + LSTM	CICIoT2023, Malimg	~ 97.00	No	Yes	Not deployed on lightweight edge devices

2.4 Research gaps and challenges

As seen in the existing literature review, several challenges and gaps form the basis for the MedIDS solution.

First, existing IDS technologies fail to provide an adequate security solution in the IoMT environment due to their limited nature and inability to respond to zero-day attacks. Although the use of machine learning and deep learning algorithms provides better accuracy in detecting attacks, most researchers focus only on one attack detection approach, either network traffic analysis or malware image classification, without implementing a hybrid IDS approach [16].

Second, deploying lightweight IDS models on the edge IoMT devices presents another significant challenge. Many IDS models based on machine learning algorithms require substantial computational power, exceeding the capacity of IoMT gateways [11].

Thirdly, most available hybrid IDS techniques have been tested on general-purpose computing platforms, and their performance has not yet been tested on low-powered edge devices such as the Raspberry Pi [16].

Fourth, there has been limited research that integrates network traffic analysis techniques with image-based malware classification within a unified, lightweight, real-time framework operating on edge devices. In [17], the authors agree that integrating an IoMT IDS system capable of detecting malware in images at the edge remains an open challenge.

All these gaps together demonstrate the need for an efficient, lightweight, hybrid, edge-based intrusion detection system capable of analyzing network traffic and malware images simultaneously and generating real-time alerts in IoMT applications. This research proposes MedIDS, which meets all these requirements by using a lightweight Decision Tree model for network-based Intrusion Detection and a lightweight MobileNetV2 model for malware image classification, and is deployed on a Raspberry Pi edge device, to support real-time intrusion detection and alert generation in the IoMT healthcare environments.

3. Research Methodology

This chapter presents the methodology of the MedIDS solution. The proposed system is designed to enhance security for IoMT devices across the healthcare environment by detecting both network intrusions and Image-based malware classification. The system runs on a Raspberry Pi edge device through an integrated interface that provides real-time traffic visualization and malware image scanning. The chapter will first describe the overall system architecture of the proposed MedIDS system. It will then describe the data description and preprocessing, machine learning and deep learning models development and evaluation, and finally, the Validation Approach and Experimental Setup.

3.1 System Architecture

MedIDS is a lightweight multi-module intrusion detection framework illustrated in Figure 2. The framework includes two different types of intrusion detection techniques: a network-based intrusion detection system for real-time network traffic monitoring and an image-based intrusion detection system for malware image classification.

The network IDS continuously monitors and analyzes IoMT traffic in real time and classifies the traffic as either attack or benign using the Decision Tree (DT) classifier, which is a ML model trained on the CICIOMT2024 dataset. The process starts with real-time network traffic capture. Then, NFStream extracts traffic features, which are then mapped to the features used during training on the CICIOMT2024 dataset to ensure the feature format consistency between the real-time data and the model. The processed features are then forwarded to the trained DT classifier for prediction. When a suspicious traffic is detected, an immediate real-time security alert is generated within the MedIDS interface, and an email notification is sent to the IT admin and medical staff.

The image-based intrusion detection system detects the presence of malicious patterns using a MobileNetV2 model trained on the MaleX dataset. The operation begins when a user uploads an image sample through the MedIDS interface, and then the model predicts whether the uploaded sample is benign or malware. If malware is detected, a security alert will be triggered in the interface, and an email notification will automatically be sent to the IT admin and medical staff.

The intrusion detection tasks are processed locally at the edge device, rather than relying on centralized cloud processing, to reduce latency, minimize the transmission of sensitive healthcare data, and enhance real-time responsiveness. The Raspberry Pi Zero 2W is selected as the edge device due to its low power consumption and suitability for resource-constrained healthcare environments, enabling both IDS modules to operate locally and present results via the Streamlit interface. Figure 2 illustrates the overall architecture of MedIDS.

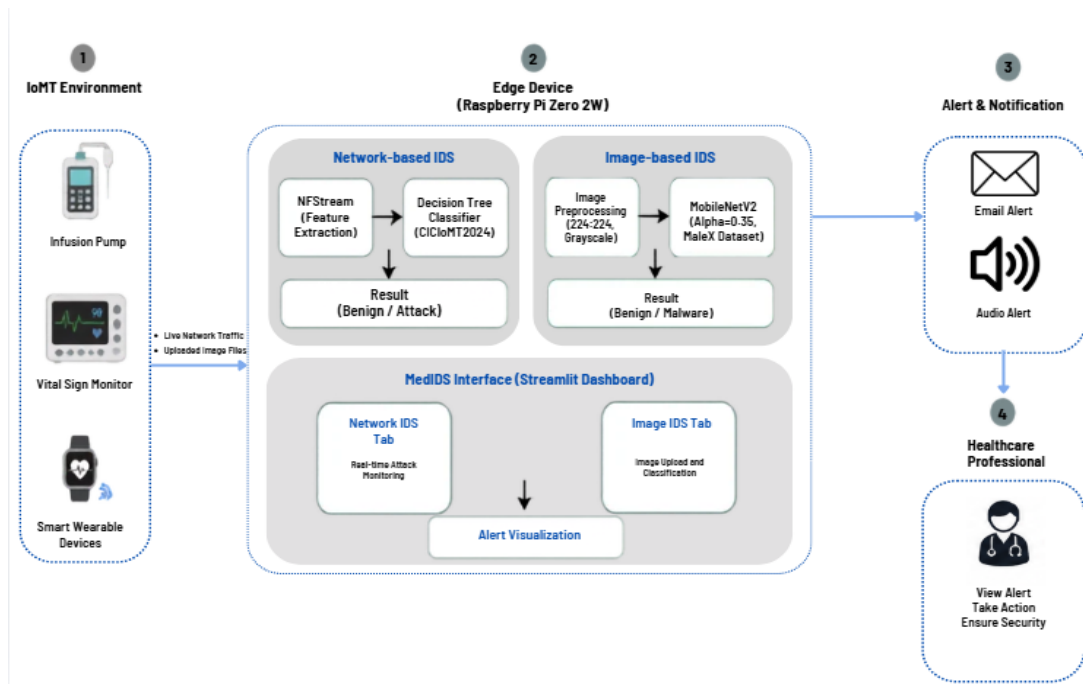


Figure 2. Overall architecture of the proposed MedIDS framework

3.2 Data Description and Preprocessing

Developing effective machine learning and deep learning models for Internet of Medical Things (IoMT) intrusion detection requires representative datasets containing both benign and malicious samples. Therefore, the CICIoMT2024 dataset was used for network traffic intrusion detection, while the MaleX dataset supported malware image classification. However, most datasets have duplicates, outliers, or missing values that require the use of many preprocessing techniques like data cleaning, balancing, and feature engineering. The following is the description and the preprocessing methods applied to the selected datasets.

3.2.1 Description and Preprocessing Methods of CICIoMT2024 Dataset

Dataset Description

CICIoMT2024 is one of the datasets developed by the Canadian Institute for Cybersecurity (CIC) in 2024. The dataset collected information from 25 actual and 15 virtual devices covering many healthcare devices such as blood pressure monitors, pulse oximeters, thermometers, glucometers, infusion pumps, sleep monitors, heart rate sensors, ECG/EKG devices, and emergency alert systems. The testbed architecture was categorized based on communication protocol to capture realistic multi-protocol network behavior with MQTT, Wi-Fi, and Bluetooth devices. To capture all network traffic with high fidelity using hardware-based packet duplication without affecting network performance, a network tap was placed between the switch and the Internet of Medical Things (IoMT) devices.

The dataset contains five major attack categories: Distributed Denial of Service (DDoS), Denial of Service (DoS), reconnaissance, MQTT-specific attacks, and spoofing. These categories include 18 distinct attack types, such as ARP spoofing, ICMP Flood, SYN Flood, TCP Flood, UDP Flood, Ping Sweep, OS Scan, Port Scan, Vulnerability Scan, MQTT Connect Flood, MQTT Publish Flood, and MQTT Malformed Data. Furthermore, CICIoMT2024 defines device lifecycles through four operational phases (power on/off, idle, active, and interaction) and three communication environments (physical, LAN, and WAN). This approach allows the dataset to represent realistic Internet of Medical Things (IoMT) device behaviour across their entire lifecycle. The dataset is comprised of several CSV files that have been organized according to attack scenarios and device types. The traffic features are collected from the PCAP files using the tool named CICFlowMeter and contain flow-based features including packet counts, byte counts, flow duration, inter-arrival times, packet length features, protocol details, and TCP/IP flag indicators [18].

Dataset Selection Rationale

The CICIoMT2024 dataset was selected for several reasons. First, the dataset is specifically designed for IoMT environments. Unlike general-purpose IoT datasets, CICIoMT2024 accurately reflects the unique traffic patterns of healthcare devices, such as periodic physiological readings, remote monitoring, and emergency alerts. Second, this dataset

represents the most recent multi-protocol security dataset available for research purposes, including the traffic of healthcare devices in Wi-Fi, MQTT, and BLE. This comprehensive coverage reflects the heterogeneous communication landscape present in actual healthcare networks. Third, device profiling throughout multiple lifecycle phases improves model training by capturing normal behaviour in various operational conditions. Finally, the CICIoMT2024 dataset is public and has been successfully used in several recent IDS papers as a benchmark for testing IoMT security systems. Overall, the CICIoMT2024 dataset is the best choice for training an IDS module based on a machine learning approach to identify network-based attacks.

Data Preprocessing

For MedIDS, CICIoMT2024 was used to build the network-based intrusion detection system model. The dataset was split into 80% for training and 20% for testing. All CSV files were combined into a unified dataset, and then each record was labeled as either benign or attack traffic, based on its file category. The binary labelling mechanism was chosen because the MedIDS network intrusion detection system was designed to detect the existence of an attack rather than classifying the specific attack family, which meets the requirements of a lightweight system.

To solve class imbalance issues in the dataset, where the attack category had more samples than benign traffic, under-sampling was applied to the majority classes, improving classification performance and preventing the model from biasing toward the majority class. After balancing, label encoding was also performed, converting categorical traffic labels into numerical values for ML models to understand.

Feature engineering was also done through feature selection and normalization. Feature selection was carried out using the SelectKBest approach with the ANOVA F-score technique to select the best features for intrusion detection. This process reduced the dataset's dimensionality and computational overhead, enabling efficient deployment on lightweight edge devices. Second, feature normalization was performed using the StandardScaler technique, where features were normalized to have mean values of zero and variance of one. As a result, features with higher numerical values do not dominate others with lower numerical values during model training. The pre-processed dataset was prepared to train and evaluate the network-based IDS models. Figure 3 illustrates the CICIoMT2024 dataset preprocessing workflow.



Figure 3. CICIoMT2024 Dataset Preprocessing Workflow

3.2.2 Description and Preprocessing Methods of MaleX Dataset

Dataset Description

MaleX is a large-scale malware dataset created in 2021 that includes 1,044,349 Windows Portable Executables (PE), divided into 864,669 malware samples and 179,725 benign samples. Malware binaries were obtained from public repositories such as VirusShare and Offensive Computing. Benign samples were selected from verified Windows system files. Each binary was analysed using VirusTotal; samples with a detection ratio of 90% or higher were classified as malware, while those with a ratio of 10% or lower were classified as benign. Only files with the MIME type "application/x-dosexec" were included to ensure that all samples were valid Windows executables. Since it is diverse and extensive, the MaleX dataset

can be considered an appropriate benchmark in designing and testing image-based malware detection systems [19].

Dataset Selection Rationale

In MedIDS, MaleX was selected to train image-based IDS for several reasons. First, the dataset contains many different malware samples, covering a large range of threats. Second, validation has been done using the VirusTotal multi-engine scanner, giving high confidence in the accuracy of the malware and benign labels. Third, MaleX is much bigger in comparison with other image malware databases, thus providing better results in terms of model training and generalizing to unseen samples. Lastly, the binary classification nature of MaleX supports the goals of the image-based IDS, which requires the ability to classify image samples with MobileNetV2.

Data Preprocessing

In MedIDS, MaleX was used to train the image-based intrusion detection system. The wide range of images used in the dataset makes it hard for the model to train at a reasonable time; therefore, a sampling method was used, selecting 2000 images from both categories to use for training. The sampling technique was used to preserve class balance while ensuring that training is computationally efficient on resource-limited devices. Moreover, the dataset was divided into 70% for training, 15% for each testing and validation dataset.

The selected images were processed and labeled as either benign (label 0) or malware (label 1). Then resized to 224x224 pixels to ensure compatibility with the MobileNetV2 model. The image normalization process involved scaling the pixel values to range from 0 to 1, thus ensuring more efficient model training and performance. At last, TensorFlow dataset pipelines were created using batching, shuffling, mapping, and prefetching operations. These techniques improve data loading efficiency, reduce training bias, and optimize resource utilization during model training. Figure 4 illustrates the MaleX dataset preprocessing workflow.



Figure 4. MaleX Dataset Preprocessing Workflow

3.3 Lightweight Machine Learning and Deep Learning Development

Machine Learning and Deep Learning technologies are widely used in IDS because of their ability to learn complex patterns from different datasets and make accurate predictions. IoMT generates complex network data, indicating that such models are ideal for intrusion detection at the IoMT node. However, it is challenging to choose the appropriate model due to the huge number of existing models, especially when it comes to lightweight models [4]. This section presents the lightweight models used in the proposed MedIDS.

3.3.1 Network-Based ML Models

Several lightweight models were trained and evaluated for the network-based intrusion detection system. These models included Decision Tree (DT), Logistic Regression (LR), and Naïve Bayes (NB), which were chosen for their low computational requirements and suitability for lightweight solutions.

Decision Tree Model

A decision tree (DT) is a supervised machine learning classification method based on decision rules. It is made up of nodes and branches; each node represents feature conditions, while each branch is a possible decision leading to the predicted class.

The model recursively splits the feature space based on decision rules. It is very interpretable, can handle non-linearity, and automatically selects informative features during the splitting process. The DT model was chosen because it allows faster classifications, low computational cost, and efficient real-time intrusion detection on Raspberry Pi [20].

Logistic Regression Model

Logistic Regression (LR) is a probability-based classification technique that predicts the output using the Sigmoid function, which outputs a probability value between 0 and 1. The model predicts the relationship between the input variables and the output variable through a series of coefficient weights learned using the maximum likelihood estimation process. It is widely used in healthcare and security sectors for binary classification problems due to its simplicity, interpretability, and computational efficiency. LR was chosen due to its robust performance in binary classification problems, minimum computational complexity involved in the process of training and testing, and probability score that can be further tuned with a desired sensitivity threshold [20].

Naïve Bayes Model

Naïve Bayes (NB) classifier is another probability model based on Bayes' theorem, which assumes independence between traffic features. The process involves calculating the posterior probability of the classes based on the prior probability and the likelihood of the features, and then choosing the one with maximum probability. NB was chosen because of the low prediction time and memory requirements, making it ideal for limited IoMT systems.[20].

The models were trained on processed network traffic features and evaluated using supervised classification techniques that distinguished between attack and benign traffic. The performance of various models was analysed using the accuracy, precision, recall, F1 score, and confusion matrix analysis. Based on the evaluation results, the decision tree model achieved the highest detection performance compared to other machine learning models and was therefore selected for deployment in the Raspberry Pi edge device.

3.3.2 Image-Based DL Models

The MobileNetV2 model was chosen for the image-based IDS because of its lightweight architecture and suitability for edge computing environments.

The model is a lightweight Convolutional Neural Network (CNN) designed for image classification tasks. It used a depth-wise separable convolution, reducing the number of parameters and computational operations compared to standard convolutional layers while still maintaining excellent classification results.

Transfer learning was used in this study to achieve good results in classifying malware images with pretrained ImageNet weight initialization. Then, this model was fine-tuned using the MaleX dataset for classification. This process saves time and enhances classification performance through the reuse of previously learned visual features [21].

In the MedIDS architecture, MobileNetV2 was trained, validated, and tested using the MaleX dataset to classify malware and benign image samples. These samples were pre-processed using resizing and normalization techniques before training to ensure compatibility with the inputs of MobileNetV2. The performance of the proposed model was evaluated using multiple classification metrics, such as accuracy, precision, recall, F1 score, and confusion matrix analysis, to determine its ability to detect malware attacks. It was chosen for its lightweight architecture, efficient image processing capability, fast inference speed, and suitability for TensorFlow Lite deployment on Raspberry Pi edge devices. Unlike the network-based IDS, only MobileNetV2 was evaluated for image classification, because it is specifically designed for lightweight edge deployment, making it well-suited to the resource and time constraints of this study.

3.4 Validation Approach and Experimental Setup

This section outlines the validation methodology used to assess the effectiveness of the proposed MedIDS framework. It details the performance evaluation metrics, edge-device performance measurements, and the experimental environment for model development, testing, and deployment.

3.4.1 Performance Evaluation Metrics

The same evaluation metrics have been used for both Network-based IDS and image-based IDS models. The models were evaluated using four different metrics: accuracy, precision, recall, and F1 score. These metrics depend on True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN), which are obtained from the confusion matrix. Equations (1), (2), (3), and (4) define the evaluation metrics [22].

Accuracy refers to the proportion of correctly classified samples, both normal and attack traffic, relative to the total number of samples.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

Precision is the proportion of correctly predicted attacks to the total predicted attacks.

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

Recall refers to the model's efficiency in predicting attacks among all attack samples.

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

The F1-score is the harmonic mean of precision and recall, providing a metric that includes both false positives and false negatives.

$$F1 \text{ score} = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (4)$$

In addition, a confusion matrix analysis was conducted to offer detailed insights into classification errors by examining true positive (TP), true negative (TN), false positive (FP), and false negative (FN) values.

3.4.2 Edge Device Performance Measurements

As the intended system architecture is meant for lightweight edge computing, there is a need to evaluate the architecture in terms of CPU utilization, memory usage, and detection latency on the Raspberry Pi Zero 2W [22].

- CPU utilization measures the percentage of processor resources used by the IDS while operating.
- Memory usage measures the amount of RAM in terms of megabytes consumed by the framework while performing the IDS tasks.
- Detection latency is the time the IDS requires to generate a prediction after receiving an input sample.

During testing, CPU utilization and memory usage metrics were obtained through Python system monitoring libraries and Raspberry Pi operating system monitoring tools. For network-based IDS, Detection Latency was measured from the completion of feature extraction by NFStream until the Decision Tree model outputs a prediction. For the image-based IDS, detection latency was measured once an image sample was uploaded until the output result was obtained by the MobileNetV2 model.

To obtain reliable measurements, repeated predictions were conducted for each of the IDS models. The average readings for CPU usage, memory usage, and latency were calculated and reported. The measurement results show the real-time capabilities of the MedIDS system and its suitability for deployment on low-resource IoMT edge devices.

3.4.3 Experimental Environment

The MedIDS framework was developed and evaluated using ML and DL models that allow real-time intrusion detection in IoMT networks. To conduct the experiments, the Python programming language was used along with TensorFlow, Scikit-learn, Pandas, and Numpy libraries for building, training, and analyzing the models. NFStream was implemented to capture and extract real-time network traffic flows. The proposed framework was intended for deployment on a Raspberry Pi Zero 2W edge device.

Table 2. Raspberry Pi Zero 2W deployment environment.

Component	Specification
Device	Raspberry Pi Zero 2W
CPU	Quad-core ARM Cortex-A53 @ 1 GHz
RAM	512 MB
Operating System	Raspberry Pi OS
User Interface	Streamlit

The Raspberry Pi Zero 2W was selected for its low cost, low power consumption, and suitability for lightweight edge devices. The device runs both the Decision Tree for network-based IDS and the MobileNetV2 model for malware image classification locally rather than depending in cloud services. This approach reduces latency, maintains privacy, and enables real-time intrusion detection in IoMT environments.

4. Implementation

In this chapter, the implementation process of the MedIDS system is described. Network-based and Image-based IDS have been integrated into a single user interface for implementation in an edge device. Both pipelines, the implementation scripts, the Raspberry Pi implementation, the user interface, and live alerts have been covered in the chapter.

4.1 Network-Based IDS Implementation

The process involved in the implementation of the network-based IDS consists of the following: CICIoMT2024 preprocessing, decision tree training, and NFStream network traffic monitoring. The network-based IDS is built to monitor and detect malicious behavior within the network traffic. It has been designed to be lightweight for edge implementation purposes. Feature extraction and mapping were used to ensure consistency of the captured packets and the trained CICIoMT2024 dataset features.

4.1.1 CICIoMT2024 Dataset Processing Implementation

The CICIoMT2024 dataset was downloaded from Kaggle and stored in Google Drive. For the preprocessing workflow, Python was used with several libraries and machine learning tools, including Pandas, NumPy, Scikit-learn, Matplotlib, and Joblib.

Initially, the dataset contained multiple distributed CSV training and testing files. For better understanding, visualization, and training, each training and testing CSV file was merged into a single dataset, and random sampling was used to reduce the dataset size to 100,000 samples for training and 30,000 samples for testing, to allow better training while preserving the overall traffic distribution. Moreover, to make the models learn from a supervised dataset, traffic labels “Benign” or “attack” were automatically assigned to each row based on file category, where benign files were labelled as Benign, and all other files with different attack types were labelled as Attack.

```

test_path = '/content/drive/MyDrive/csv/test'
test_files = os.listdir(test_path)
test_list = []

for file in test_files:
    if file.endswith('.csv'):
        file_path = os.path.join(test_path, file)
        temp_df = pd.read_csv(file_path)
        temp_df['label'] = 'Benign' if 'benign' in file.lower() else 'Attack'
        test_list.append(temp_df)

test_df = pd.concat(test_list, ignore_index=True)
test_df = test_df.sample(n=30000, random_state=42)
print("test shape :", test_df.shape)

+++ test shape : (30000, 46)

train_path = '/content/drive/MyDrive/csv/train'
train_files = os.listdir(train_path)
train_list = []

for file in train_files:
    if file.endswith('.csv'):
        file_path = os.path.join(train_path, file)
        temp_df = pd.read_csv(file_path)
        temp_df['label'] = 'Benign' if 'benign' in file.lower() else 'Attack'
        train_list.append(temp_df)

train_df = pd.concat(train_list, ignore_index=True)
train_df = train_df.sample(n=100000, random_state=42)
print("train shape :", train_df.shape)

+++ train shape : (100000, 46)

```

Figure 5.CSV merging, traffic labelling, and dataset sampling.

The CICIoMT2024 dataset had more attack samples than benign samples, which may bias machine learning models toward the majority class, resulting in low accuracy and more false alerts. Figure 6 shows the original class distribution in the training dataset, where attack samples outnumber benign samples. Therefore, to avoid such errors, the majority attack class was randomly reduced to match the number of benign samples using random under-sampling. Figure 7 illustrates that both classes contained an equal number of samples, which reduces the risk of model bias during training.

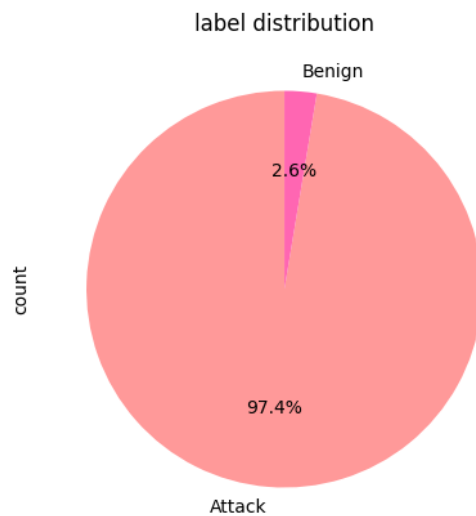


Figure 6.Class distribution before under-sampling

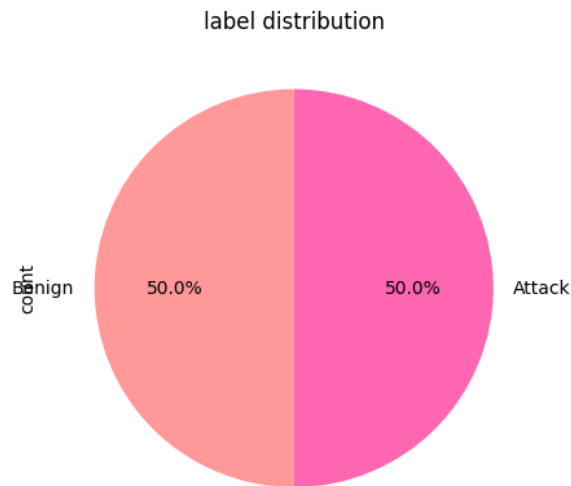


Figure 7. Class distribution after under-sampling

```
benign_df = train_df[train_df['label'] == 'Benign']
attack_df = train_df[train_df['label'] == 'Attack']

print(len(benign_df))
print(len(attack_df))

min_size = min (len(benign_df), len(attack_df))

attack_Sample = attack_df.sample(n=min_size, random_state=42)

train_df = pd.concat([benign_df, attack_Sample])
train_df = train_df.sample(frac=1, random_state=42)
```

Figure 8. Under-sampling implementation

After that, label encoding was implemented using the LabelEncoder method to convert the categorical traffic labels (“Benign” and “Attack”) to numerical values (0 and 1) for the machine learning models to understand.

```
le = LabelEncoder()
train_df['label'] = le.fit_transform(train_df['label'])
test_df['label'] = le.transform(test_df['label'])

print(train_df['label'].value_counts())
```

Figure 9. Implementation of label encoding

The preprocessing workflow also includes separating numerical features from labels before the training process, where features are the input variables for the machine learning models, and labels are the target variables. Furthermore, only numerical features were used for model training to ensure compatibility with the chosen machine learning algorithms. Features were aligned between the training and testing datasets to ensure both contain the same feature structure to avoid matching errors.

```
x_train = train_df.drop(columns=['label'], axis=1)
x_train = x_train.select_dtypes(include=['int64', 'float64'])
y_train = train_df['label']

x_test = test_df.drop(columns=['label'], axis=1)
x_test = x_test.select_dtypes(include=['int64', 'float64'])
y_test = test_df['label']

x_train, x_test = x_train.align(x_test, join='inner', axis=1)
```

Figure 10. Feature-label separation and feature alignment implementation.

For feature engineering techniques, Feature selection and normalization were applied. The feature selection technique used the SelectKBest method with ANOVA F-score analysis to select the 20 most significant features associated with intrusion detection and reduce irrelevant features. This helped to reduce computational overhead with lightweight deployment on the edge device. After selecting the associated features, the StandardScaler method was used for feature normalization to standardize feature values into a similar range before training to improve performance and stability. Finally, the processed data sets were prepared to train and test the network- based intrusion detection system. Figure 11 shows the 20 most relevant features selected by the ANOVA F-score method for model training.

```
selector = SelectKBest(score_func=f_classif, k=20)
x_train_selected = selector.fit_transform(x_train, y_train)
x_test_selected = selector.transform(x_test)
selected_features = x_train.columns[selector.get_support()]
print(selected_features)

Index(['Header_Length', 'Rate', 'Srate', 'syn_flag_number', 'rst_flag_number',
      'psh_flag_number', 'ack_flag_number', 'rst_count', 'HTTPS', 'TCP',
      'UDP', 'ICMP', 'Tot sum', 'Max', 'AVG', 'Std', 'Tot size', 'Magnitue',
      'Radius', 'Variance'],
      dtype='object')

scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train_selected)
x_test_scaled = scaler.transform(x_test_selected)
```

Figure 11. Implementation of feature selection and feature normalization

4.1.2 Machine Learning Models Training

The Scikit-learn library in Python was used for training three lightweight ML models for the network-based IDS: Decision Tree (DT), Logistic Regression (LR), and Naïve Bayes (NB). Each model was trained multiple times to figure out the best training approaches with the best performance, first using the original network traffic features, then with selected features, and finally with normalized features.

To train a decision tree, the function `DecisionTreeClassifier` was applied, with hyperparameters `max_depth=10` to prevent overfitting by controlling the tree size, `min_samples_leaf=10` to ensure more stable decision nodes, and `random_state=42` for achieving reproducibility of the experimental results. To train a Naïve Bayes classifier, the `GaussianNB` classifier was used. For logistic regression, the `LogisticRegression` classifier was used, with the hyperparameter `max_iter=1000` to increase the maximum number of iterations during training and ensure proper convergence. The experimental design enabled direct comparison of feature selection and normalization effects on model performance.

```
tree_model = DecisionTreeClassifier(max_depth=10, min_samples_leaf=10, random_state=42).fit(x_train, y_train)
tree_fs = DecisionTreeClassifier(max_depth=10, min_samples_leaf=10, random_state=42).fit(x_train_selected, y_train)
tree_scaled = DecisionTreeClassifier(max_depth=10, min_samples_leaf=10, random_state=42).fit(x_train_scaled, y_train)

nb_model = GaussianNB().fit(x_train, y_train)
nb_fs = GaussianNB().fit(x_train_selected, y_train)
nb_scaled = GaussianNB().fit(x_train_scaled, y_train)

lr_model = LogisticRegression(max_iter=1000).fit(x_train, y_train)
lr_fs = LogisticRegression(max_iter=1000).fit(x_train_selected, y_train)
lr_scaled = LogisticRegression(max_iter=1000).fit(x_train_scaled, y_train)
```

Figure 12. Training implementation of Decision Tree, Naïve Bayes, and Logistic Regression models.

```
def evaluate_model(model, model_name, x_test, y_test):
    y_pred = model.predict(x_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    model_results.append({ "Model": model_name, "Accuracy": accuracy, "Precision": precision, "Recall": recall, "F1-Score": f1 })
    print(model_name)
    print(f"Accuracy : {accuracy:.4f}")
    print(f"Precision : {precision:.4f}")
    print(f"Recall : {recall:.4f}")
    print(f"F1-Score : {f1:.4f}")

    print("\nClassification Report:\n")
    print(classification_report(y_test, y_pred))

    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Benign', 'Attack'])
```

Figure 13. Evaluation function

After training the models, an evaluation function was implemented to calculate accuracy, precision, recall, and F1-score metrics and to generate confusion matrices for each model, to help find the best model associated with the MedIDS criteria.

4.1.3 Real-time Network Traffic Monitoring and Detection

For Real-time network traffic monitoring and detection, MedIDS used Python and the NFStream library. The script was designed to monitor live network traffic and classify it as either benign or an attack. The process includes real-time traffic capturing, feature extraction, feature mapping, traffic preprocessing, and attack prediction.

Several libraries were imported: NFStream for flow extraction, Pandas for data manipulation, Joblib for loading the model and pipeline, and NumPy for numerical processing. First, joblib was used to load the trained DT model, the preprocessing pipeline, and the selected features used during training.

```
from nfstream import NFStreamer
import pandas as pd
import joblib
import numpy as np

pipeline = joblib.load('models/Net-basedIDS3.pkl')
```

Figure 14. Importing libraries and loading the trained DT model

NFStream is used to capture real-time traffic flows from the wireless network interface (wlan0) connected with the Raspberry Pi Zero 2W device and enables statistical flow analysis to extract flow-based traffic statistics required for intrusion detection. NFStream was configured with `statistical_analysis=True` to enable flow feature extraction. An active timeout of 10 seconds and an idle timeout of 5 seconds were configured to support continuous flow monitoring and timely traffic processing.

```
streamer = NFStreamer(source= "wlan0", statistical_analysis=True, active_timeout=10, idle_timeout=5)
```

Figure 15. NFStream real-time traffic monitoring

The feature mapping function was implemented to maintain compatibility between live traffic and the same 20 trained dataset features. Several traffic features were calculated, including packet rate, traffic size, packet statistics, protocol indicators, and flow characteristics.

```
def features_mapping(flow):
    try:
        duration_sec = flow.bidirectional_duration_ms / 1000.0
        if duration_sec <= 0 :
            duration_sec = 0.000001
        total_packets = flow.src2dst_packets + flow.dst2src_packets
        total_bytes = flow.src2dst_bytes + flow.dst2src_bytes
        data = {
            'Header_length': flow.bidirectional_bytes,
            'Rate' : total_packets / duration_sec,
            'Srate' : flow.src2dst_packets / duration_sec,
            'syn_flag_number' : 0,
            'rst_flag_number' : 0,
            'psh_flag_number' : 0,
            'ack_flag_number' : 0,
            'rst_count' : 0,
            'HTTPS': 1 if flow.dst_port == 443 else 0,
            'TCP': 1 if flow.protocol == 6 else 0,
            'UDP': 1 if flow.protocol == 17 else 0,
            'ICMP': 1 if flow.protocol == 1 else 0,
            'Tot sum': total_packets,
            'Max': total_packets,
            'AVG': total_bytes / max(total_packets, 1),
            'Std': 0,
            'Tot size': total_bytes,
            'Magnitue': total_bytes,
            'Radius': 0,
            'Variance': total_packets,
        }

        return data

    except Exception as e:
        print("error:", e)
```

Figure 16. Feature mapping function

Each extracted traffic flow was then transformed into a Pandas DataFrame and aligned with the feature structure chosen for the training process. The features were then standardized using the StandardScaler function before classification. To predict the probability of malicious behavior for each captured flow, the `predict_proba()` function was used. The predicted probabilities generated by the model were evaluated against a predefined decision threshold to determine whether each traffic flow was benign or malicious.

4.2 Image-Based IDS Implementation

The implementation of the image-based IDS focused on developing a lightweight malware image classification system using the MaleX dataset and MobileNetV2 model to classify malware and benign image samples. This section discusses image preprocessing, dataset preparation, TensorFlow dataset pipeline construction, MobileNetV2 model training, and malware image prediction.

4.2.1 MaleX Dataset Processing Implementation

The MaleX dataset was downloaded from Kaggle and stored in Google Drive. For the preprocessing workflow, Python was used with several libraries, including TensorFlow, NumPy, Matplotlib, PIL, and Scikit-learn.

After downloading the dataset, it was extracted into malware and benign image folders to simplify the dataset and enable automated image loading during training and preprocessing. Next, a random sampling method was used to obtain a balanced subset containing 5000 malware images and 5000 benign images, ensuring that the dataset was unbiased during classification.

```
Benign_Dir = "/content/malex_dataset/benign"
Attack_Dir = "/content/malex_dataset/malware"

benign_files = [
    os.path.join(Benign_Dir, f)
    for f in os.listdir(Benign_Dir)
    if f.endswith(".png")
]

attack_files = [
    os.path.join(Attack_Dir, f)
    for f in os.listdir(Attack_Dir)
    if f.endswith(".png")
]

SEED = 42
random.seed(SEED)
benign_files = random.sample(benign_files, 5000)
attack_files = random.sample(attack_files, 5000)
```

Figure 17. Malware and benign image folder loading and random sampling

Further validation of the preprocessing was achieved by visualizing randomly selected malware and benign images through Matplotlib. This approach helps validate the accuracy of the image loading process and ensures proper categorization of the dataset before training.

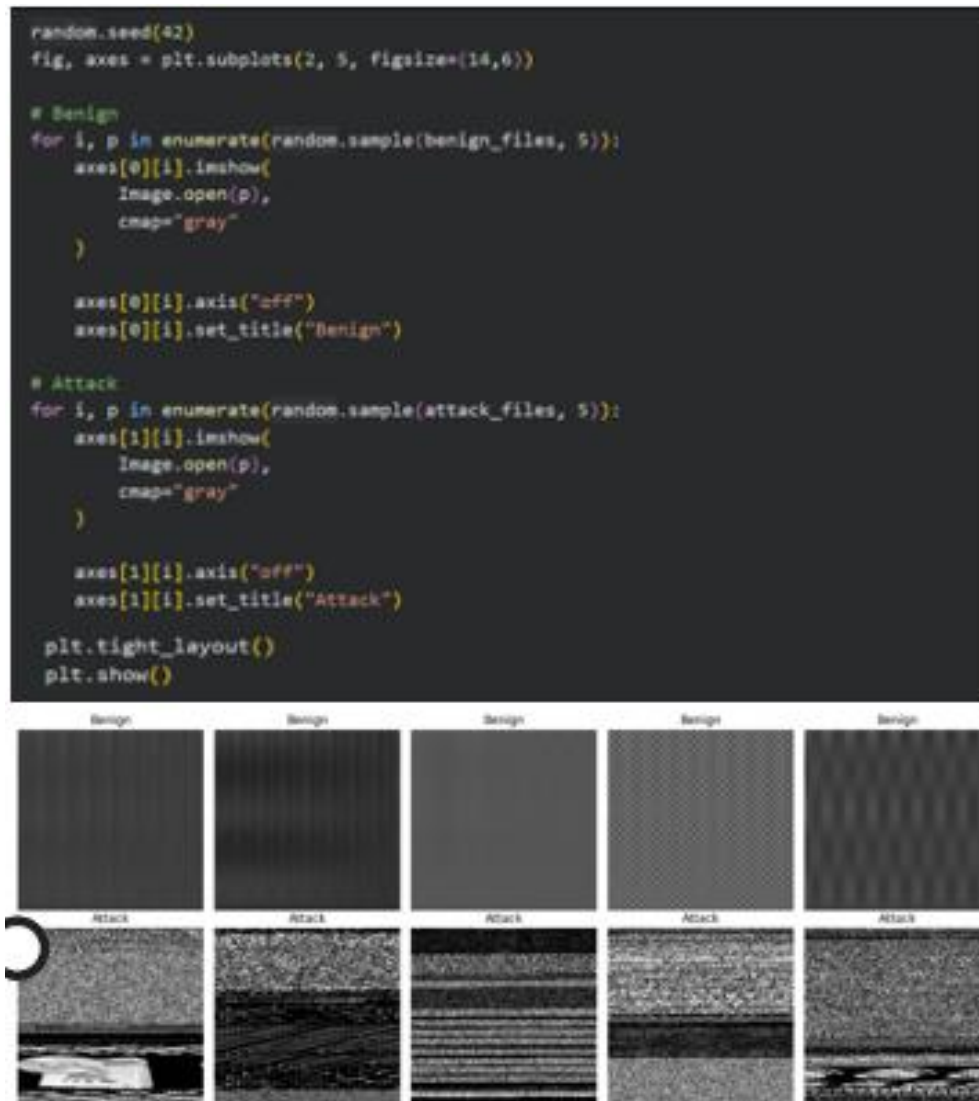


Figure 18. Visualization of malware and benign image samples using Matplotlib.

Before dataset preparation, parameters like image size, batch size, and random seed initialization were set. The image size was set to 224 x 224 to match the input requirement for the MobileNetV2 model. Labels were manually mapped to each category in the dataset, where benign image samples were labeled with 0, and malware images were labeled with 1. Such a labeling technique was useful for supervised malware image classification.

```

IMG_SIZE = 224
BATCH_SIZE = 32
SEED = 42

np.random.seed(SEED)
tf.random.set_seed(SEED)

benign_files = [
    os.path.join(Benign_Dir, f)
    for f in os.listdir(Benign_Dir)
    if f.endswith(".png")
]

ben_files = random.sample(benign_files, 2000)
ben_labels = [0] * len(ben_files)

attack_files = [
    os.path.join(Attack_Dir, f)
    for f in os.listdir(Attack_Dir)
    if f.endswith(".png")
]

attack_files = random.sample(attack_files, 2000)
attack_labels = [1] * len(attack_files)

all_paths = ben_files + attack_files
all_labels = ben_labels + attack_labels

```

Figure 19. Image preprocessing parameters and labeling technique initialization.

After dataset preparation, the images and labels were split into three datasets – training, validation, and testing – using the `train_test_split()` method. Where training represent 70% of the dataset, and 15% represented by each validation and testing datasets. The stratification technique was used during both splitting stages to ensure that both the malware images and benign images were distributed evenly in the training, validation, and test datasets.

```

train_paths, temp_paths, train_labels, temp_labels = train_test_split(
    all_paths,
    all_labels,
    test_size=0.30,
    stratify=all_labels,
    random_state=SEED
)
val_paths, test_paths, val_labels, test_labels = train_test_split(
    temp_paths,
    temp_labels,
    test_size=0.50,
    stratify=temp_labels,
    random_state=SEED
)

```

Figure 20. Dataset splitting

TensorFlow datasets were used with batching, shuffling, mapping, and prefetching techniques. Batching groups several images into batches during training, and shuffling ensures a randomized dataset to minimize training bias. Mapping techniques were applied to automate the preprocessing of images during the loading process, each image resized to 224×224 pixels, converted into a tensor representation, and normalized according to the MobileNetV2 preprocessing requirements. Prefetching was used to enhance lightweight data loading performance and reduce delays during training. The resulting TensorFlow datasets were then used as inputs for MobileNetV2 training and evaluation.

```
def build_dataset(paths, labels, shuffle=False):
    ds = tf.data.Dataset.from_tensor_slices((paths, labels))

    if shuffle:
        ds = ds.shuffle(len(paths), seed=SEED)
    ds = ds.map( process_image, num_parallel_calls=AUTOTUNE)
    ds = ds.batch(BATCH_SIZE)
    ds = ds.prefetch(AUTOTUNE)
    return ds

train_ds = build_dataset(train_paths, train_labels, shuffle=True)
val_ds = build_dataset(val_paths, val_labels)
test_ds = build_dataset(test_paths, test_labels)
```

Figure 21. TensorFlow dataset pipeline construction

4.2.2 MobileNetV2 Model Training

The MobileNetV2 model is a lightweight CNN architecture. MobileNetV2 trained using TensorFlow and Keras to perform lightweight image-based IDS. The model can distinguish between malware and benign images while maintaining efficient training.

The model adopted a pretrained ImageNet model to enhance feature extraction performance and reduce the training complexity. The MobileNetV2 base model was fine-tuned with its pretrained layers left trainable, enabling the network to adapt ImageNet features to the MaleX malware image dataset. The input images were resized to $224 \times 224 \times 3$ pixels to match MobileNetV2 requirements. The parameter `include_top=False` was used to remove the default classifier layer and replace it with a custom lightweight malware classification layer. Moreover, the parameter `alpha=0.35` was used to minimize the model's size and compute resources, thus making the architecture more appropriate for deployment on lightweight edge devices. The

Global Average Pooling technique was applied to reduce the dimensionality of the extracted feature maps before classification. After that, a dense layer with 128 neurons and ReLU activation was used to enhance malware feature learning ability. A dropout layer was added to minimize overfitting, with a rate of 0.3. Moreover, a sigmoid activation function was applied to the output layer for performing binary classification between malware and benign image samples.

```
def build_model():
    inputs = layers.Input(shape=(224,224,3))
    base_model = MobileNetV2(alpha= 0.35, input_shape=(224,224,3), include_top=False, weights="imagenet")
    base_model.trainable = True
    x = base_model(inputs)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(128, activation="relu")(x)
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(1, activation="sigmoid")(x)
    model = models.Model(inputs, outputs)

    return model

model = build_model()
model.summary()
```

Figure 22. MobileNetV2 model architecture with a custom classification layer

The MobileNetV2 model was compiled with the Adam optimizer with a learning rate of 0.001 to provide more stable optimization during the training process. Binary cross-entropy was chosen as the loss function because the malware image classification task is a binary classification problem. The accuracy, recall, precision, and F1-score were used as additional parameters for monitoring malware detection efficiency during training.

```
model.compile( optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3), loss="binary_crossentropy",
    metrics=
    [tf.keras.metrics.BinaryAccuracy( name="accuracy"),
    tf.keras.metrics.Recall(name="recall"),
    tf.keras.metrics.Precision(name="precision") ]
)
```

Figure 23. Model compilation and metrics initialization

Some callback mechanisms were used in the training process to increase model stability and avoid any unnecessary computations. The ModelCheckpoint callback saved the best-performing model based on validation accuracy. The ReduceLROnPlateau callback automatically reduced the learning rate when validation accuracy stopped improving. Moreover, the EarlyStopping callback was also used to stop the training process in case no further improvement was seen in validation accuracy.

The model was trained with the TensorFlow `model.fit()` function, using the prepared training and validation datasets. The number of training epochs = 5, representing the number of training iterations over the dataset, whereas the validation set was used for evaluating the learning process and preventing overfitting. Moreover, the callback functions are used for model saving, learning rate tuning, and early stopping during training. The `verbose=1` parameter was also used to show real-time learning during training.

```
start_time = time.time()
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=5,
    callbacks=callbacks,
    verbose=1
)
```

Figure 24. MobileNetV2 training using TensorFlow `model.fit()` and callback mechanisms

After training was completed, evaluation metrics were used to evaluate the performance of malware image classification using the testing dataset. The prediction probability scores were calculated using the trained model and then assigned to malware and benign image classifications. The confusion matrix and classification report methods have been utilized using Scikit-learn for evaluating the performance of malware image classification.

4.2.3 Real-time Malware Image Detection

A malware image detection script was developed using Python with NumPy, PIL, and TensorFlow Lite libraries. The script was designed to load the trained image-based IDS pipeline and classify image samples.

First, the TensorFlow Lite interpreter was imported using the `ai_edge_litert` library to load the model in TensorFlow Lite (tflite) format. Allocation of the required tensors was done by the interpreter, and then the information of the input and output tensors was extracted to perform lightweight inference on the loaded model. TensorFlow Lite was selected because it supports lightweight malware classification and reduces computational costs during edge deployment.

```
import numpy as np
from PIL import Image
from ai_edge_litert.interpreter import Interpreter
import sys
```

Figure 25. Importing libraries for image-based IDS and loading the TensorLite model

Pre-processing of the malware images occurred before prediction to ensure compatibility with the MobileNetV2 training configuration. Each image was converted to RGB format and resized to 224×224 pixels. Pixel values were then converted to NumPy arrays and normalized by scaling pixel values to the range [0,1]. Lastly, the dimensions of the image tensors were expanded for batch processing.

After pre-processing was completed, the prepared image tensor was passed into the TensorFlow Lite interpreter using the `set_tensor()` function. The `invoke()` function was then called to conduct inference for lightweight malware prediction. The prediction probability value was obtained from the output tensor and compared against a predefined threshold to determine whether the analyzed image represent benign or malicious activity.

```
def predict(image):
    img = image.convert('RGB').resize((224, 224))
    img_array = np.array(img, dtype=np.float32)
    img_array = img_array / 255.0
    img_array = np.expand_dims(img_array, axis=0)
    interpreter.set_tensor(input_details[0]['index'], img_array)
    interpreter.invoke()

    prediction = interpreter.get_tensor(output_details[0]['index'])[0][0]

    print(prediction)

    return prediction
```

Figure 26. Image preprocessing and prediction

4.3 Raspberry Pi Edge Deployment and MedIDS Interface

This section presents the deployment and implementation of the proposed MedIDS framework in the Raspberry Pi Zero 2W edge device. The deployment involved the implementation of the previously designed network-based and image-based intrusion detection modules in a lightweight edge computing setup for IoMT healthcare applications. This was achieved through the configuration of the Raspberry Pi setup, execution of the trained intrusion detection modules, analysis of lightweight resource usage, and implementation of the MedIDS user interface for intrusion detection and malware image classification. Furthermore, this section highlights how the proposed framework allows for lightweight real-time detection operations.

4.3.1 Raspberry Pi Edge Deployment Environment

The MedIDS system was deployed on a Raspberry Pi Zero 2W device to test the effectiveness of lightweight intrusion detection in the context of edge computing. The Raspberry Pi device was chosen for use due to its affordability, small size, minimal power requirements, and suitability for lightweight IoMT healthcare applications. First, the environment was prepared using the Raspberry Pi Imager software, and Raspberry Pi OS Lite (64-bit) was chosen to minimize graphical overhead. In the operating system customization process, the hostname and user login details were configured to simplify the device management process.

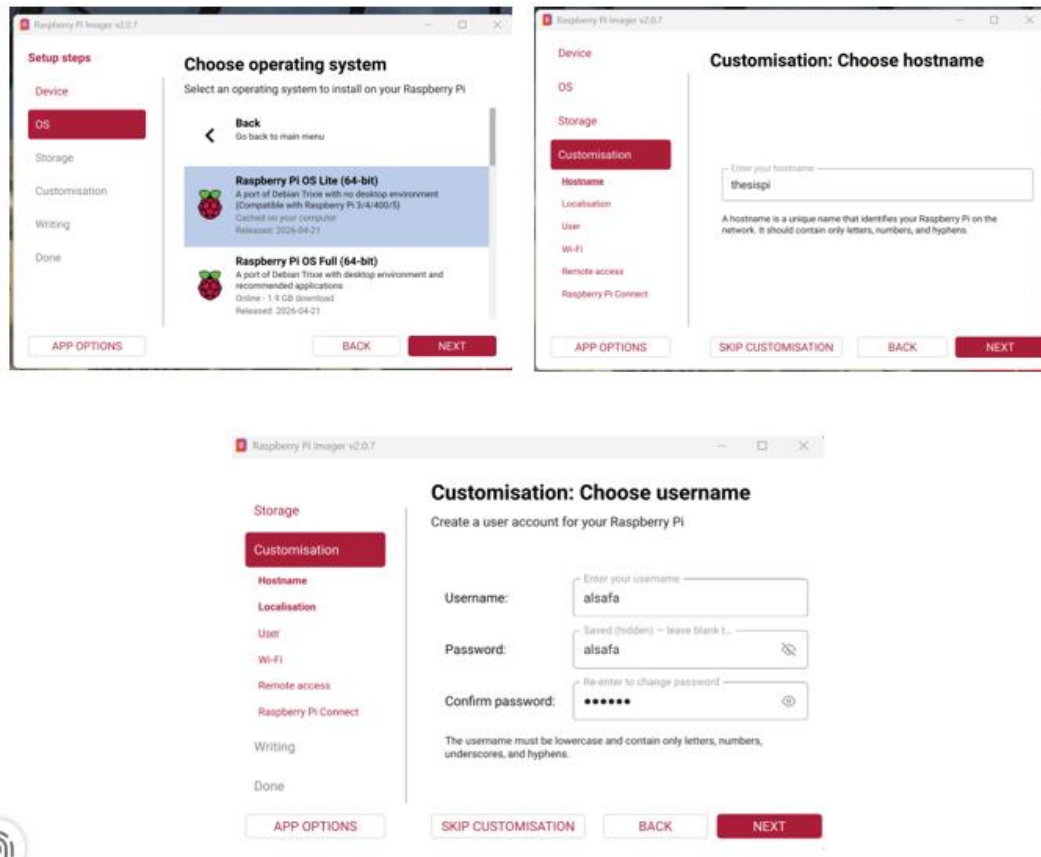


Figure 27. Raspberry Pi OS installation and device configuration

The Remote SSH access was configured to provide lightweight remote management and monitoring of the Raspberry Pi during implementation and testing, without requiring direct interaction with the Raspberry Pi hardware.

```
C:\Users\alsaf>ssh alsafa@thesispi.local
alsafa@thesispi.local's password:
Linux thesispi 6.12.75+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.12.75-1+rpt1 (2026-03-11) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon May 18 21:53:45 2026 from 172.20.10.4
alsafa@thesispi:~ $
```

Figure 28. Raspberry Pi SSH remote access

Configuration of the deployment environment was further validated using system-level verification commands to check for operating system compatibility, architecture configuration, and the presence of Python runtime. The Raspberry Pi operated on Debian GNU/Linux OS, arm64 architecture, and Python 3.13 runtime to provide compatibility with lightweight machine learning and TensorFlow Lite deployment libraries employed in the MedIDS platform.

```

alsafa@thesispi:~ $ python --version
Python 3.13.5
alsafa@thesispi:~ $ hostnamectl
  Static hostname: thesispi
        Icon name: computer
        Machine ID: 2d782746e35e4f999d1ff8b3bc0119
        Boot ID: 97c84ad169e04a749290e728295b9f
  Operating System: Debian GNU/Linux 13 (trixie)
        Kernel: Linux 6.12.75+rpt-rpi-v8
        Architecture: arm64

```

Figure 29. Raspberry Pi operating system and Python environment verification

Once the Raspberry Pi setup was completed, the two IDS modules were successfully installed and executed on the edge device for real-time intrusion detection experiments. The installation process ensured compatibility between MedIDS framework and the Raspberry Pi Zero 2W edge environment.

```

(venv) alsafa@thesispi:~/MedIDS/Network_ids $ sudo /home/alsafa/MedIDS/venv/bin/python network.py
/home/alsafa/MedIDS/venv/lib/python3.13/site-packages/sklearn/base.py:463: InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeClassifier from version 1.6.1 when using version 1.8.0. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
/home/alsafa/MedIDS/venv/lib/python3.13/site-packages/sklearn/base.py:463: InconsistentVersionWarning: Trying to unpickle estimator StandardScaler from version 1.6.1 when using version 1.8.0. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
Starting live capture...
Latency:0.00795
🔥 Attack detected !!! 🔥 Source:fe80::8096:98ff:fe61:5164 -> Destination: ff02::1

```

Figure 30. Successful execution of network-based IDS on Raspberry Pi

```

(venv) alsafa@thesispi:~/MedIDS/image_ids $ sudo /home/alsafa/MedIDS/venv/bin/python image.py
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.

```

Figure 31. Successful execution of image-based IDS on Raspberry Pi

4.3.2 MedIDS Interface

For user-friendly interaction with MedIDS, a simple User Interface (UI) was created using the Streamlit framework in Python. The interface integrates both network-based IDS and image-based IDS components into a unified platform, enabling healthcare and IT professionals to respond quickly to intrusions. It provides real-time intrusion detection, malware image classification, alerts, and lightweight interaction for a Raspberry Pi environment. Streamlit was chosen for its lightweight structure, fast deployment, and ability to work with machine learning applications.

```
(venv) alsafa@thesispi:~/MedIDS $ sudo /home/alsafa/MedIDS/venv/bin/streamlit run myUI.py

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.

2026-05-20 11:50:21.011 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://172.20.10.11:8501
External URL: http://188.140.143.231:8501

INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
```

Figure 32. Streamlit-based MedIDS application running on Raspberry Pi

The main interface is separated into two Streamlit tabs :Network IDS and Image IDS. The Network-IDS tab was designed to handle real-time IoMT traffic monitoring, while the Image-IDS tab was implemented for classifying malware images. The sidebar component was used to provide general information about the MedIDS system. This design provides a unified environment while maintaining a simple and user-friendly workflow.

The Network IDS interface had real-time traffic monitoring capabilities through the interactive Start Monitoring and Stop Monitoring controls. When the user clicks start monitoring, the interface runs the script that performs the network intrusion detection using Python subprocess. The live network traffic is constantly captured and analyzed through NFStream flow extraction, and the intrusion detection outputs are shown through the interface in real time. This allows for real-time monitoring and visualization of suspicious IoMT network activities.

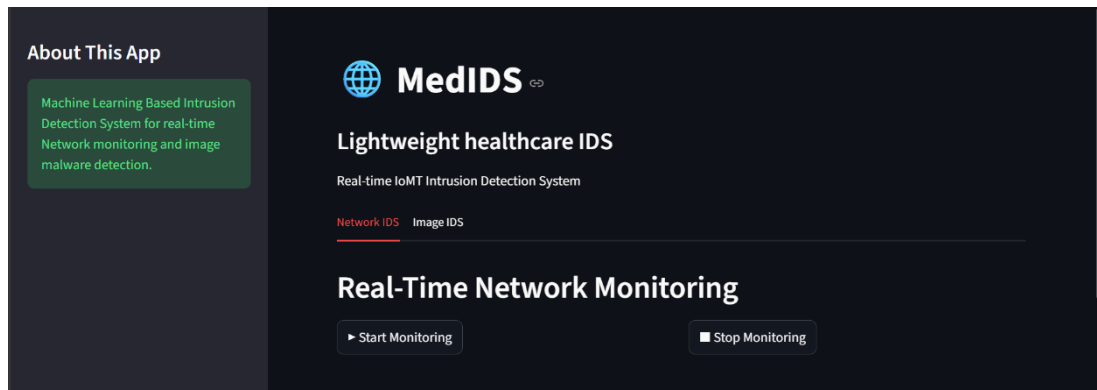


Figure 33. Network-based IDS monitoring interface

The Image IDS tab allows users to upload images through the application. Once an image is uploaded, it displays within the interface and is passed to the TensorFlow Lite MobileNetV2 model, which outputs the prediction used for classification.

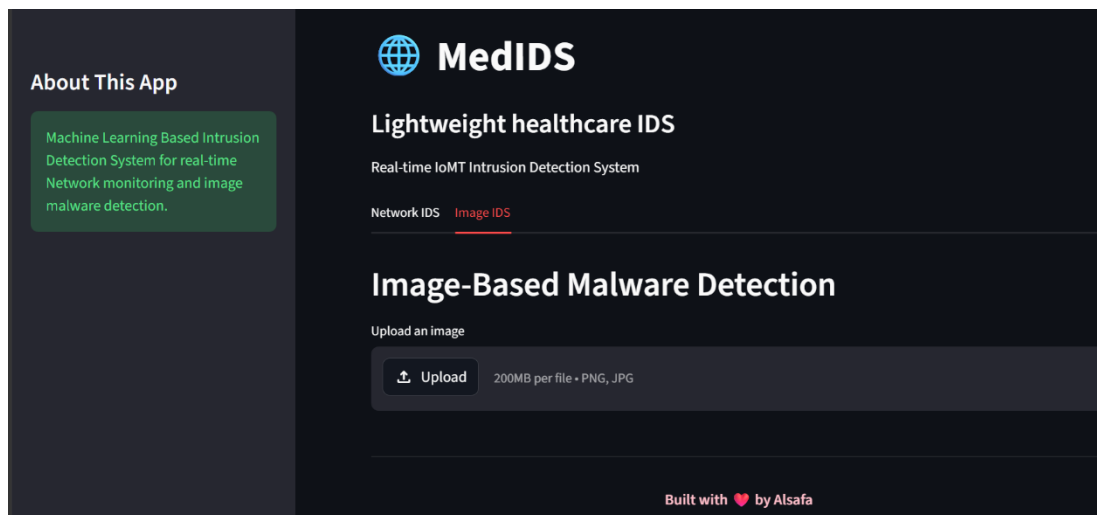


Figure 34. Image-based malware detection interface

The MedIDS interface adopted several lightweight alerting and notification systems during intrusion detection operations to notify users of potential threats. Whenever suspicious network traffic or malware is detected, MedIDS generates real-time warning alerts using Streamlit toast messages and visual error components. Moreover, an audio-based alert is triggered automatically, using HTML audio alerts. Email notifications were also used through the smtplib library and Gmail SMTP services in Python to ensure remote intrusion monitoring.

The complete implementation of the MedIDS framework, including model training scripts, deployment files, and user interface components, is available in the GitHub repository provided in Appendix A.

5. Results and Discussion

This chapter presents the evaluation and performance of MedIDS. The performance of both the network and image IDS was evaluated, with a discussion of the models used and their lightweight resource utilization. Finally, real-time attack simulation experiments were conducted to evaluate MedIDS's ability to detect attacks and malware images in real time.

5.1 Performance of Network-based Intrusion Detection System

This section focuses on evaluating the performance of the network-based IDS used in MedIDS. Three different machine learning models were evaluated in terms of accuracy, precision, recall, and F1 score, and analysed using a confusion matrix. Moreover, lightweight edge performance was measured in terms of CPU utilization, memory consumption, and detection latency on the Raspberry Pi Zero 2W edge device.

5.1.1 Machine Learning Models Evaluation

Decision Tree (DT), Logistic Regression (LR), and Naïve Bayes (NB) are the lightweight ML models evaluated to determine the most suitable one for the network-based IDS edge deployment. Each model was trained and evaluated based on three preprocessing approaches: the original network traffic features, feature selection, and finally, feature normalization or scaling. The decision tree with feature selection was determined as the most suitable option among all the evaluated models.

Decision Tree

The Decision Tree (DT) model using the original network traffic features had an accuracy of 99.21%, a precision of 75.35%, a recall of 98.87%, and a F1 score of 85.52%. The confusion matrix shows that 29067 benign samples were correctly classified as benign and are known as True Negatives (TN), while 697 attack samples were also classified correctly as attacks, and they are known as True Positives (TP). In addition, there are 228 benign samples classified as attacks, representing False Positives (FP), and only eight attack samples were

classified as benign traffic, representing False Negatives (FN). The high recall score shows that the model successfully identifies most of the attack traffic samples with a small number of missed attacks since it had a very low FN score. However, the low precision score compared to recall was caused by a relatively large number of false positives, where some benign traffic samples were identified as attack traffic samples, causing false alarms. However, for intrusion detection cases, recall is more important than other metrics because missing an attack traffic is generally more critical than generating additional alerts.

The other preprocessing approaches showed similar performance measures with almost no significant difference in the overall performance of the original Decision Tree. However, although feature selection did not improve performance, the deployment pipeline retained it, maintaining consistency in the preprocessing between the training and real-time inference stages of MedIDS. Additionally, the pipeline helped in reducing features while providing a lightweight intrusion detection system for the Raspberry Pi Zero 2W edge device.

```
evaluate_model(tree_model, "decision tree ", x_test, y_test)

decision tree
Accuracy : 0.9921
Precision : 0.7535
Recall : 0.9887
F1-Score : 0.8552
```

Figure 35. Performance evaluation of the Decision Tree model using original features.

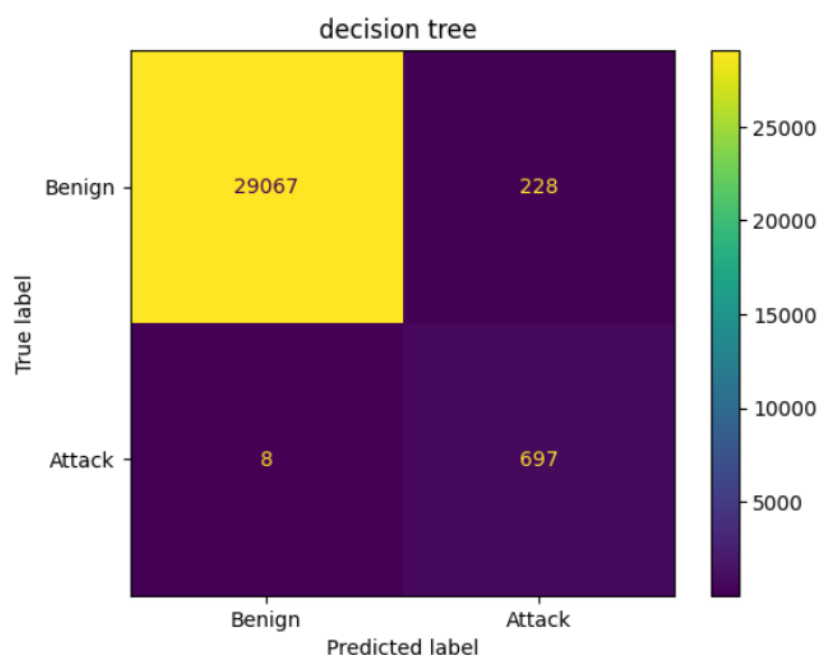


Figure 36. Confusion matrix of the Decision Tree model using original features.

```

evaluate_model(tree_fs, "decision tree with feature selection" , x_test_selected, y_test)

decision tree with feature selection
Accuracy : 0.9921
Precision : 0.7546
Recall : 0.9858
F1-Score : 0.8549

```

Figure 35. Performance evaluation of the Decision Tree model using feature selection.

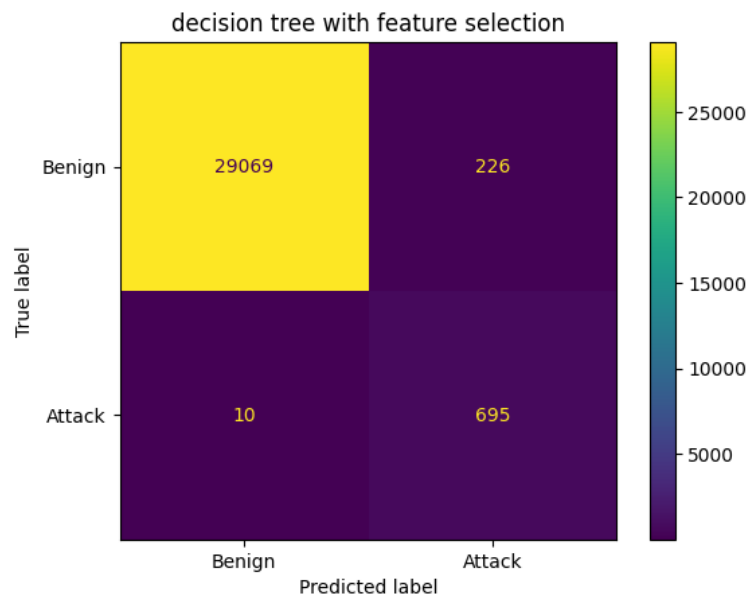


Figure 36. Confusion matrix of the Decision Tree model using feature selection.

```

evaluate_model(tree_scaled, "decision tree with scaling" , x_test_scaled, y_test)

decision tree with scaling
Accuracy : 0.9921
Precision : 0.7546
Recall : 0.9858
F1-Score : 0.8549

```

Figure 37. Performance evaluation of the Decision Tree model using feature scaling.

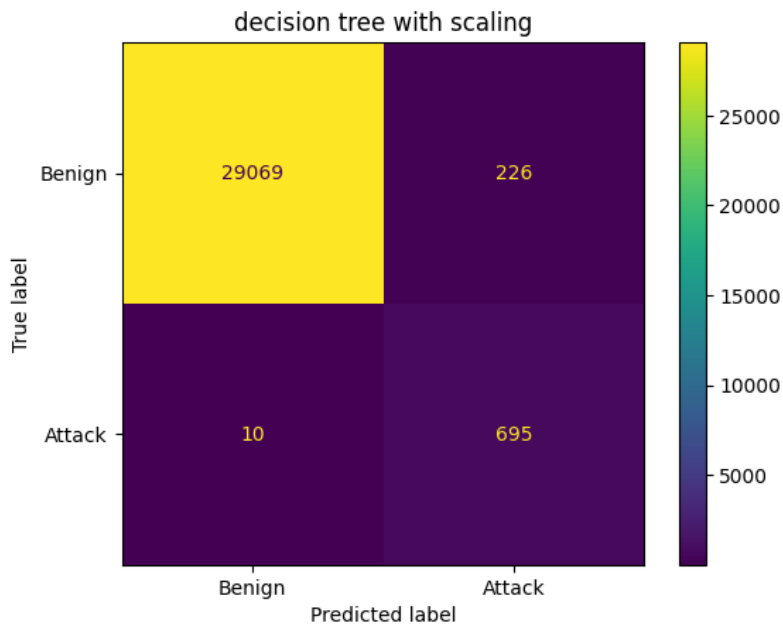


Figure 38. Confusion matrix of the Decision Tree model using feature scaling.

Naïve Bayes

The Naïve Bayes (NB) classifier using the original network traffic features achieved 97.13% accuracy, 45.01% precision, 99.72% recall, and 62.02% F1-score. From the confusion matrix, 28,436 benign traffic samples were correctly classified as True Negatives (TN), while 703 of attack traffic samples were correctly classified as attack traffic as True Positives (TP). In addition, 859 benign traffic samples were incorrectly classified as attacks, representing False Positives (FP), while two attack traffic samples were incorrectly classified as benign traffic, representing False Negatives (FN). The recall score is very high for Naïve Bayes classification since almost all attack traffic samples were correctly classified. However, its low precision rate was a result of the high number of False Positives (FP), increasing unnecessary alerts.

```
evaluate_model(nb_model, "naive bayes ", x_test, y_test)

naive bayes
Accuracy : 0.9713
Precision : 0.4501
Recall : 0.9972
F1-Score : 0.6202
```

Figure 39. Performance evaluation of the Naïve Bayes model using original features.

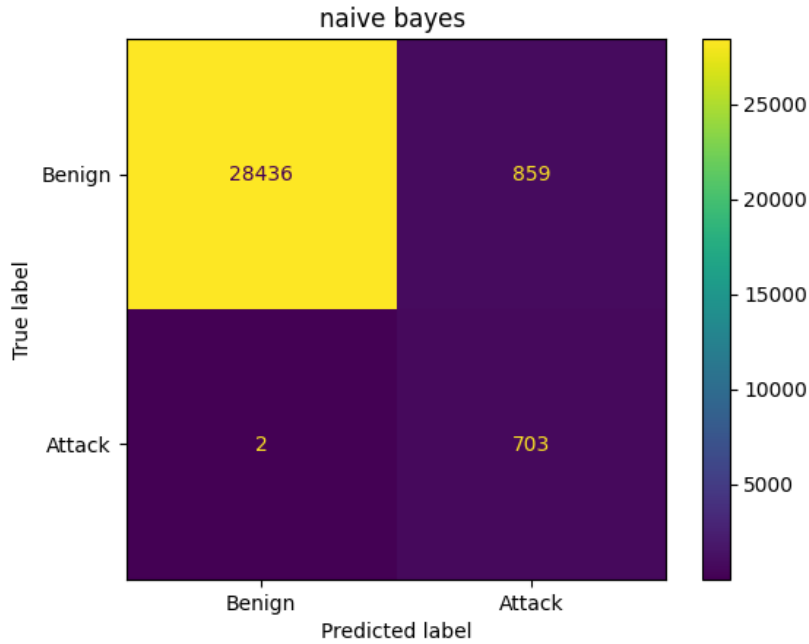


Figure 40. Confusion matrix of the Naïve Bayes model using original features

Using feature selection with Naïve Bayes resulted in an accuracy of 98.33%, precision of 59.71%, recall of 88.51%, and F1-score of 71.31%, showing fewer false positives, better precision, and decreasing unnecessary false alarms compared to the model trained with original features. However, false negatives increased, reducing the overall recall performance and causing more attack traffic samples to be missed. After applying feature scaling, the performance increased with an accuracy of 98.37%, precision of 59.26%, recall of 97.59%, and F1-score of 73.74%. Moreover, the number of False Negatives (FN) decreased, making the Naïve Bayes algorithm more accurate in detecting attacks.

Despite these improvements, the Naïve Bayes classifier remained less effective comparing to decision tree model. It achieved higher recall but generate more false positives resulting in less precision and F1-score, which made it less suitable for lightweight real-time applications on MedIDS.

```

evaluate_model(nb_fs, "naive bayes with feature selection", x_test_selected,y_test)

naive bayes with feature selection
Accuracy : 0.9833
Precision : 0.5971
Recall : 0.8851
F1-Score : 0.7131

```

Figure 41. Performance evaluation of the Naïve Bayes model using feature selection.

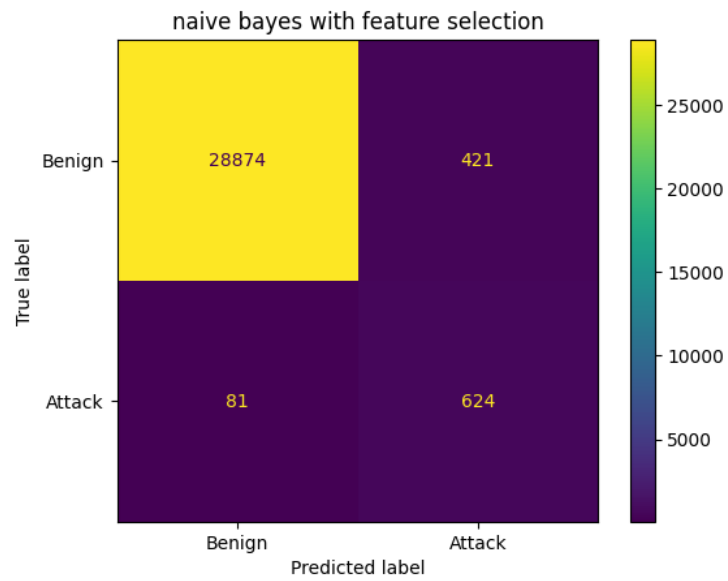


Figure 42. Confusion matrix of the Naïve Bayes model using feature selection

```
evaluate_model( nb_scaled,"Naive Bayes with Scaling",x_test_scaled,y_test)
```

```
Naive Bayes with Scaling
Accuracy : 0.9837
Precision : 0.5926
Recall    : 0.9759
F1-Score  : 0.7374
```

Figure 43. Performance evaluation of the Naïve Bayes model using feature scaling.

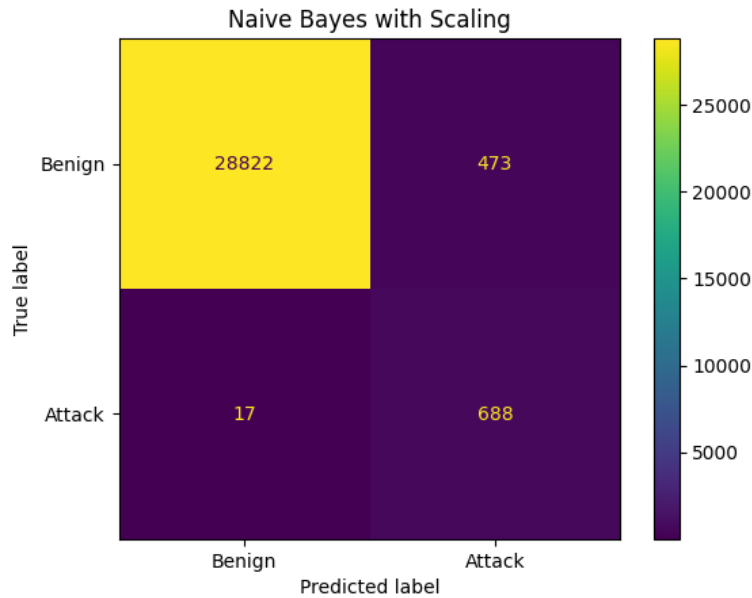


Figure 44. Confusion matrix of the Naive Bayes model using feature scaling.

Logistic Regression

The Logistic regression (LR) classifier using the original network traffic features achieved 98.30%, 58.98%, 90.35% and 71.37% on the accuracy, precision, recall, and F1-score metrics, respectively. The confusion matrix analysis determined that there were 28,852 benign traffic samples correctly identified, representing True Negatives (TN), and 637 attack samples were correctly identified as attack, representing True Positives (TP). Also, there were 443 benign traffic samples that the model misclassified as attacks (FP) and 68 attack traffic samples that the model misclassified as benign (FN). The Logistic Regression classifier demonstrates a balanced classification performance with moderate precision and recall values. However, it generates more FP and FN compared to the Decision Tree model, reducing overall attack detection ability and increasing false alarms.

```
evaluate_model(lr_model, "logistic regression", x_test, y_test)

logistic regression
Accuracy : 0.9830
Precision : 0.5898
Recall : 0.9035
F1-Score : 0.7137
```

Figure 45. Performance evaluation of the Logistic regression model using original features.

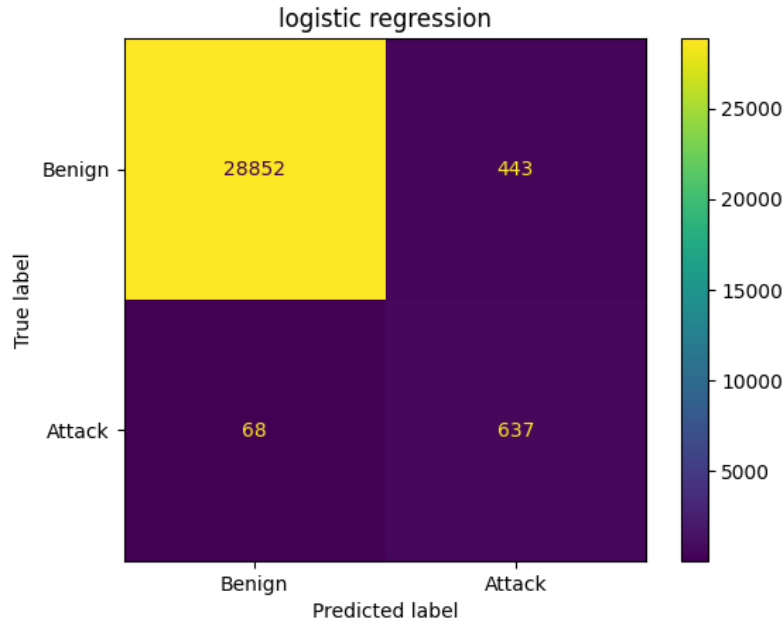


Figure 46. Confusion matrix of the Logistic Regression model using original features.

The use of the feature selection technique with the Logistic Regression classifier achieved 98.48% accuracy, 61.97% precision, 91.06% recall, and 73.75% F1-score, demonstrating a small improvement in classification performance and reduction in the rate of False Positives (FP). However, when using feature scaling, the classifier achieved 98.43% accuracy, 59.97% precision, 99.43% recall, and 74.81% F1-score. Moreover, there was a significant reduction in the rate of false negatives (FN) after performing feature scaling. Nevertheless, the Logistic Regression classifier obtained lower classification performance when compared to the Decision Tree classifier. Although feature selection and feature scaling improved recall and F1-score values, the classifier still generated more classification errors, making it less preferable for edge deployment.

```

> evaluate_model(lr_fs, "logistic regression with feature selection", x_test_selected, y_test)
• logistic regression with feature selection
  Accuracy : 0.9848
  Precision : 0.6197
  Recall    : 0.9106
  F1-Score  : 0.7375

```

Figure 47. Performance evaluation of the Logistic regression model using feature selection.

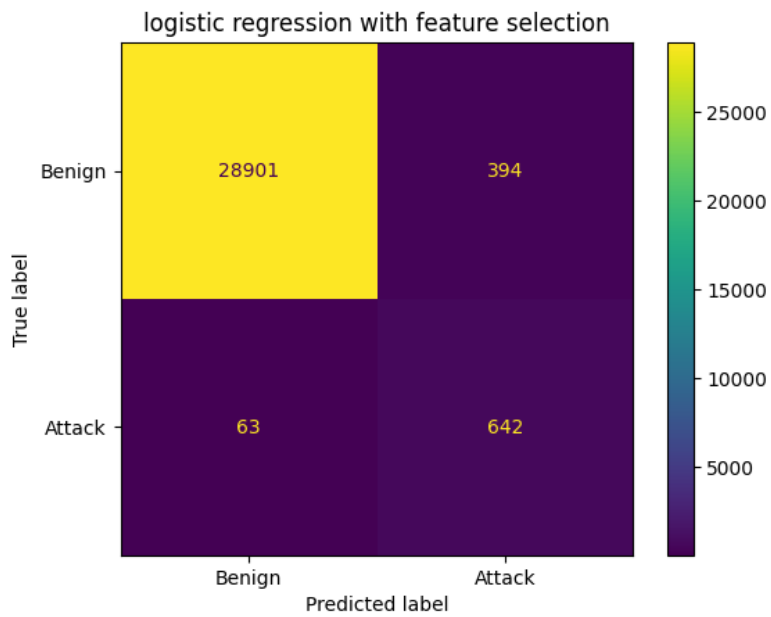


Figure 50. Confusion matrix of the Logistic Regression model using feature selection.

```
evaluate_model(lr_scaled, "logistic regression with scaling", x_test_scaled, y_test)

logistic regression with scaling
Accuracy : 0.9843
Precision : 0.5997
Recall : 0.9943
F1-Score : 0.7481
```

Figure 48. Performance evaluation of the Logistic regression model using feature scaling.

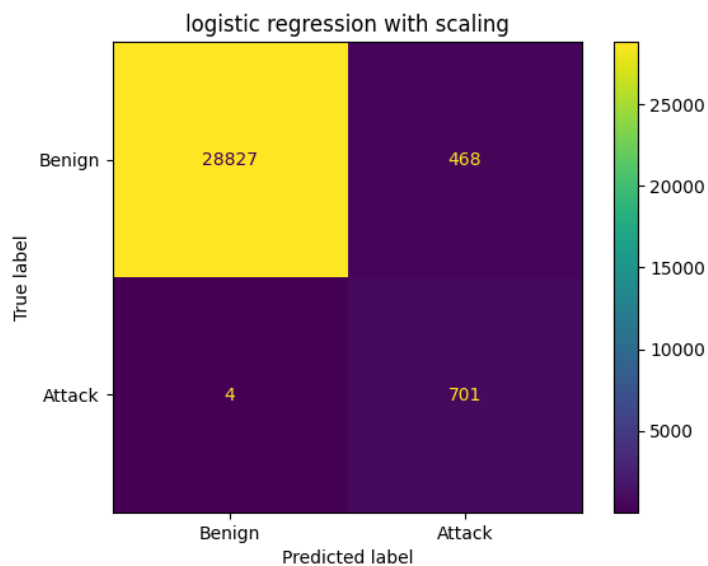


Figure 52. Confusion matrix of the Logistic Regression model using feature scaling.

Comparison between the evaluated models is shown in Table 3 and Figure 53. The Decision Tree has achieved the best performance among all other models, obtaining the highest accuracy and F1-score while maintaining a high recall rate and the lowest number of false positives.

The Naïve Bayes had a very high recall score compared to other models, which indicates its capability to detect most attacks correctly, but with less precision, meaning that many benign traffic samples were incorrectly classified as attacks, resulting in more false alarms. The Logistic Regression classifier showed more balanced performance than Naïve Bayes, but its detection capability is lower than that of the decision tree.

The application of feature selection and feature scaling led to a slight enhancement in the classification performances of both Naïve Bayes and Logistic Regression classifiers, mainly by minimizing the False Negative (FN) rates and enhancing recall measures. However, in the Decision Tree, these techniques produce few changes.

Considering the requirements of lightweight edge deployment, strong attack detection, and low false alarms, the Decision Tree model was selected for network-based IDS.

Table 3. Performance comparison of network-based IDS machine learning models.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Decision Tree	99.21	75.35	98.87	85.52
Decision Tree with Feature Selection	99.21	75.46	98.58	85.49
Decision Tree with Scaling	99.21	75.46	98.58	85.49
Naïve Bayes	97.13	45.01	99.72	62.02
Naïve Bayes with Feature Selection	98.33	59.71	88.51	71.31
Naïve Bayes with Scaling	98.37	59.26	97.56	73.74
Logistic Regression	98.3	58.98	90.35	71.37

Logistic Regression with Feature Selection	98.48	61.97	91.06	73.75
Logistic Regression with Scaling	98.43%	59.97	99.43	74.81

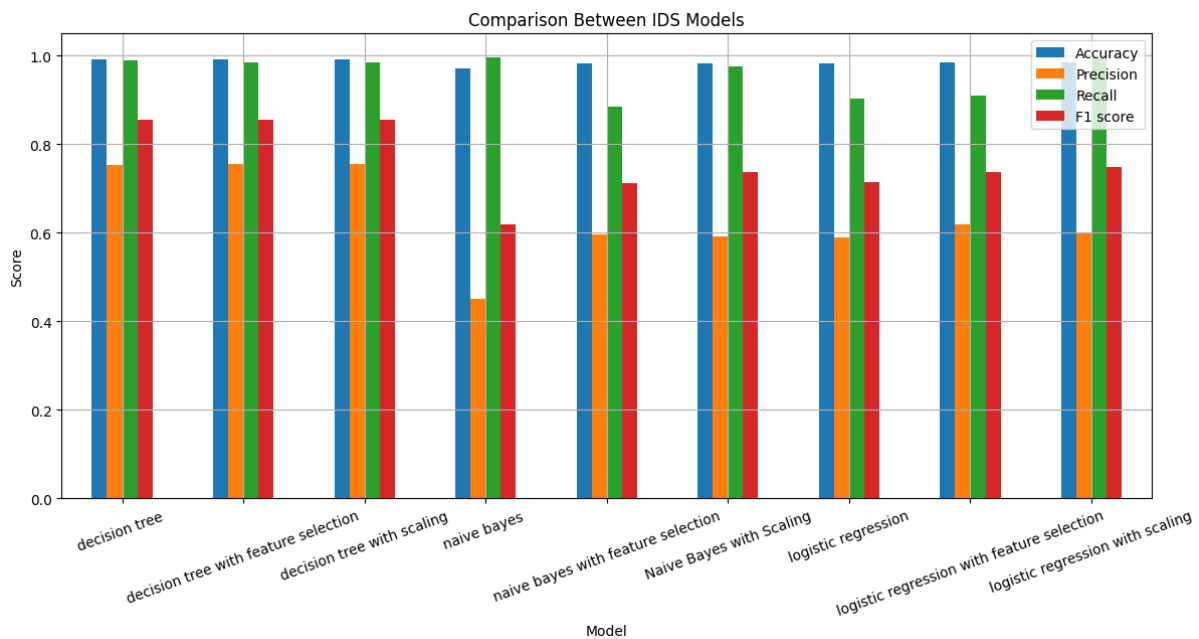


Figure 49. Comparison of accuracy, precision, recall, and F1 score among evaluated network-based IDS machine learning models.

5.1.2 Raspberry Pi Edge Deployment Performance of the Network-based IDS

The lightweight implementation of the network-based IDS is evaluated after deploying MedIDS in the Raspberry Pi Zero 2W. The evaluation was conducted based on three metrics: CPU usage, memory usage, and detection delay. This assessment was carried out to find whether the IDS would be effective in lightweight IoMT environments.

Before conducting any intrusion detection processes, the Raspberry Pi Zero 2W device showed extremely low idle resource consumption with approximately 155 MB of memory and

little CPU usage. This shows the idle performance of the deployment environment before the MedIDS monitoring process.

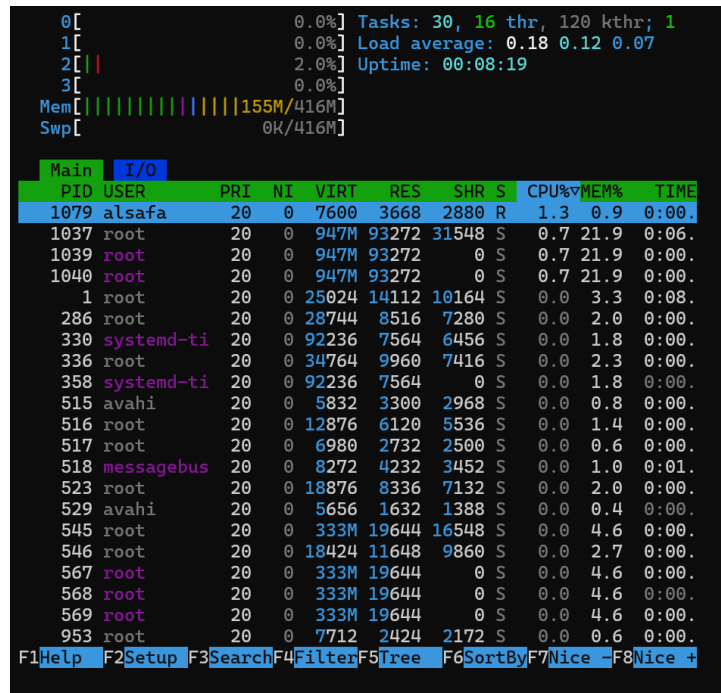


Figure 50. Resource utilization of Raspberry Pi Zero 2W before IDS monitoring.

During the benign traffic monitoring process, the network-based IDS managed to detect and analyze IoMT traffic flows using NFStream, while simultaneously conducting intrusion detection using Decision Trees in the Streamlit user interface window. This integration of Streamlit, NFStream, and the lightweight Decision Tree classifier was able to perform well in terms of execution time and memory usage, with about 216 MB memory utilization and low CPU utilization. Additionally, the intrusion detection latency measured during regular monitoring was around 0.00123 seconds (1.23 ms), indicating a fast real-time intrusion detection capability in the Raspberry Pi edge device.

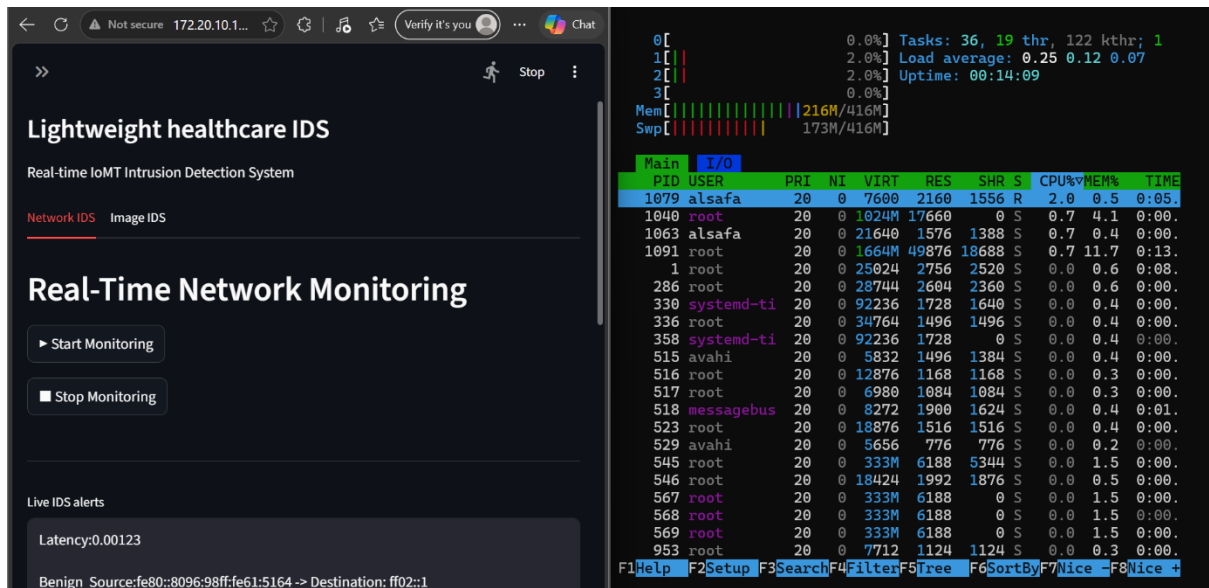


Figure 51. Resource utilization during benign traffic monitoring.

When generating an attack targeting the Raspberry Pi, the CPU utilization increased due to the large volume of generated packets, which the IDS must continuously inspect, process, and classify a very large number of incoming packets and network flows in real time. In addition, memory utilization increased to around 321 MB without exceeding the resources of the Raspberry Pi device. The latency in the intrusion detection process during attack monitoring was around 0.03971 seconds (39.71 ms).

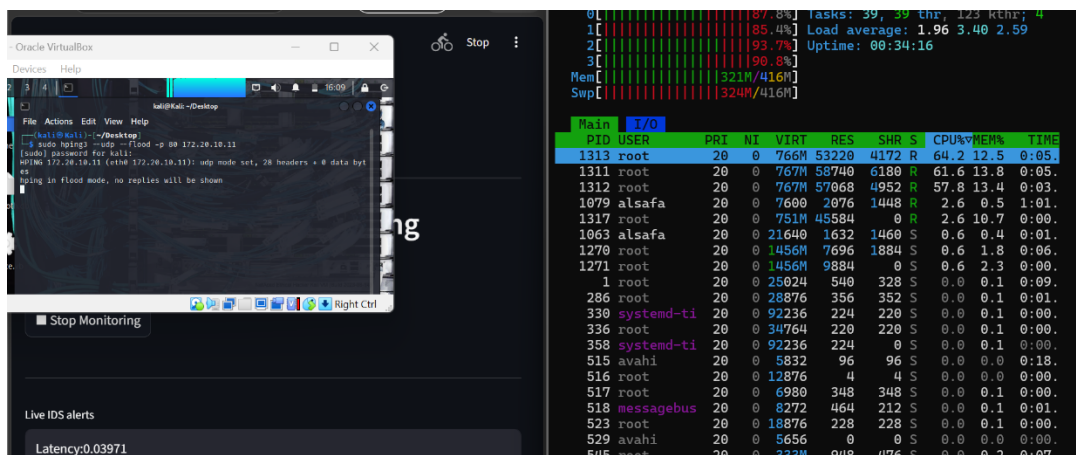


Figure 52. Resource utilization during attack traffic monitoring.

The Network-based IDS maintained low resource utilization and fast detection latency. However, regarding the increase in resource consumption during attacks, the system continues to work within the Raspberry Pi Zero 2W limits. This shows that the MedIDS framework works well for real-time intrusions in IoMT devices with limited resources.

Table 4. Raspberry Pi Zero 2W resource utilization of the network-based IDS.

Scenario	CPU Usage (%)	Memory Usage (MB)	Detection Latency (S)
Idel State	~1	155	-
Benign Traffic Monitoring	Low <5	216	0.000123
Attack Traffic Monitoring	Increased	321	0.03971

5.2 Performance of Image-based Intrusion Detection System

This section evaluates the performance of the image-based IDS implemented in MedIDS using the lightweight MobileNetV2 deep learning model. This includes the assessment of malware image classification performance, confusion matrix analysis, and lightweight inference suitability for Raspberry Pi deployment environments.

MobileNetV2 was chosen as the main deep learning model for detecting malware images due to the time limitation and lightweight requirements of the deployment environment.

5.2.1 MobileNetV2 Model Evaluation

The MobileNetV2 model was selected and evaluated to determine how well it can classify malware and benign image samples. The evaluation process was conducted through model architecture analysis, training performance analysis, validation performance analysis, and classification result analysis based on accuracy, precision, recall, F1-score, and confusion matrix analysis.

The model showed continuous improvement through the training epochs. The accuracy increased from 51.67% in the first epoch to 75.83% in the final epoch. The final evaluation results achieved 76.33% for accuracy, 100% for precision, 52.67% for recall, and 69.00% for F1-score. The confusion matrix results indicate that there are 300 images of benign samples identified correctly as benign, representing True Negatives (TN), and 158 malware images

identified correctly as malware, representing True Positives (TP). In addition, no benign images were misclassified as malware images, which means zero False Positives (FP), and this is why the precision value is so high, showing that the malware predictions made by the classifier were completely accurate. However, 142 malware images were misclassified as benign images known as False Negatives (FN), which affected the recall score because some malware images were not detected. This may be due to the lightweight architecture of MobileNetV2 and the smaller dataset used for edge-oriented training. The model was selected because deployment efficiency was prioritized over maximum classification accuracy.

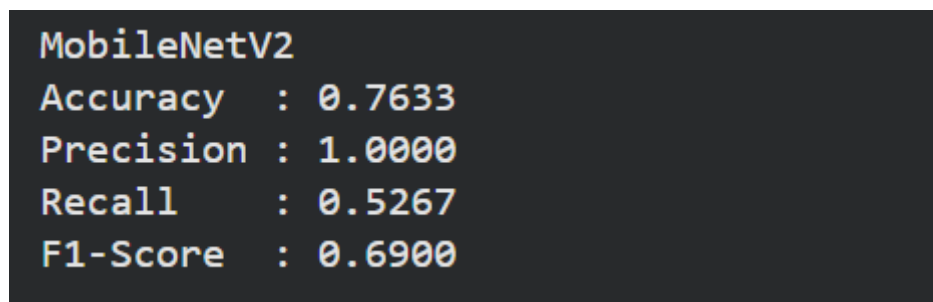


Figure 53. Performance evaluation of the MobileNetV2 model.

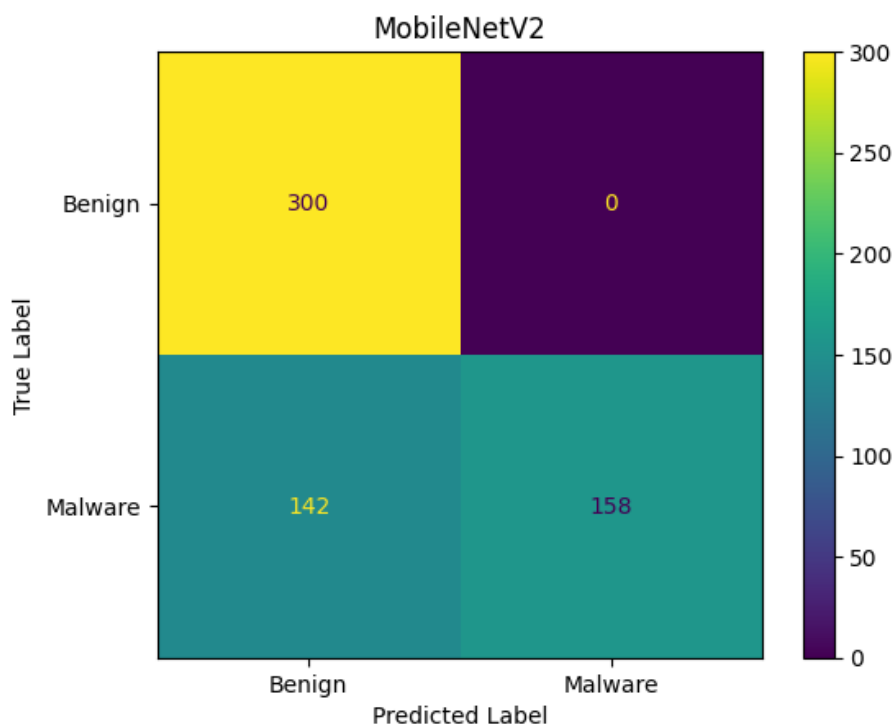


Figure 54. Confusion matrix of the MobileNetV2 malware image classifier.

5.1.2 Raspberry Pi Edge Deployment Performance of the Image-based IDS

The lightweight implementation of the image-based IDS is evaluated after deploying MedIDS in the Raspberry Pi Zero 2W. This assessment was carried out to find whether the IDS would be effective in lightweight IoMT environments.

The IDS detected malware image samples via the Streamlit interface using the TensorFlow Lite MobileNetV2 model and ran efficiently on the Raspberry Pi Zero 2W. When a benign image is uploaded, the prediction process shows that 164 MB of memory is used, and CPU usage remains moderate. Moreover, the malware image inference latency time was around 0.08307 seconds.

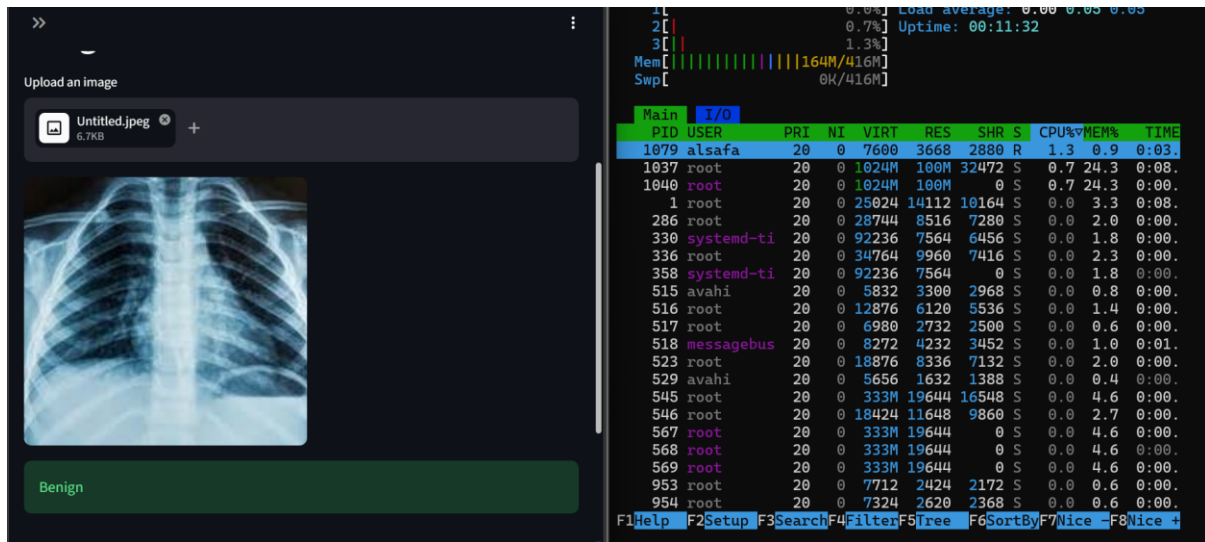


Figure 55. Resource utilization during benign image classification on Raspberry Pi Zero 2W.

When a malware image sample is detected, the deployment results indicate that the Raspberry Pi device utilized about 166 MB of memory and low CPU usage. The latency time taken for malware image processing was approximately 0.08196 seconds. The small difference in resource consumption for benign and malware predictions shows that inference performance is stable and consistent, regardless of the image classification result.

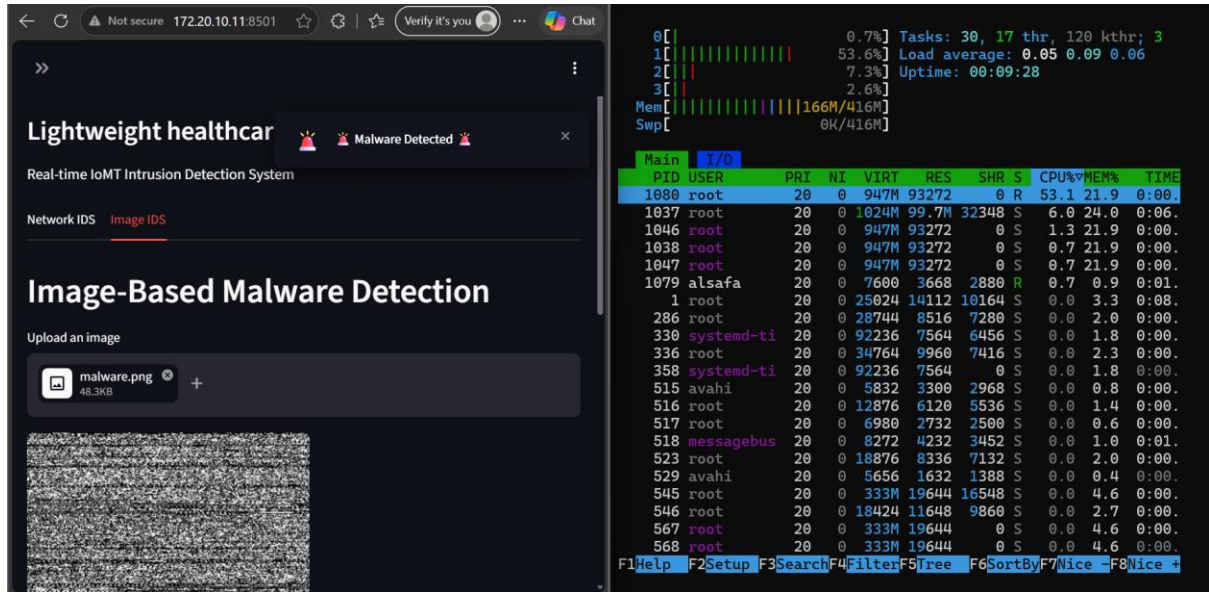


Figure 56. Resource utilization during malware image classification on Raspberry Pi Zero 2W.

The deployment results show that the image-based intrusion detection system (IDS), developed using the TensorFlow Lite MobileNetV2 model, achieved successful performance on the resource-constrained Raspberry Pi Zero 2W edge device due to its effective memory usage, minimal CPU usage, and reduced inference latency. The results support the feasibility of implementing image-based IDS in the MedIDS framework for IoMT healthcare environments.

Table 5. Raspberry Pi Zero 2W resource utilization of the image-based IDS.

Scenario	CPU Usage (%)	Memory Usage (MB)	Latency (s)
Benign Image Prediction	Low	164	0.08307
Malware Image Prediction	Low	166	0.08196

5.3 Real-Time Attack Simulation and Detection Results

This section presents the results of real-time attack simulation and detection for the MedIDS framework deployed on the Raspberry Pi Zero 2W edge device. The evaluation covered live network attack simulations and malware image upload tests using the Streamlit interface. The goal was to assess MedIDS's effectiveness in real-time intrusion detection for lightweight IoMT healthcare environments.

5.3.1 Real-Time Network Attack Detection

The real-time network-attack detection experiment aimed to test the effectiveness of the proposed IDS in detecting live network attacks in a lightweight IoMT environment. Before starting the attack, the address of the Raspberry Pi Zero 2W device was identified using the (`ip a`) Linux command. The device was configured with an IP address of 172.20.10.11, representing the attack target.

```
alsafa@thesispi:~ $ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKN
OWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_co
del state UP group default qlen 1000
    link/ether 88:a2:9e:95:c5:e3 brd ff:ff:ff:ff:ff:ff
    inet 172.20.10.11/28 brd 172.20.10.15 scope global dynamic n
oprefixroute wlan0
        valid_lft 2655sec preferred_lft 2655sec
    inet6 fe80::8aa2:9eff:fe95:c5e3/64 scope link proto kernel_l
l
        valid_lft forever preferred_lft forever
```

Figure 57. Raspberry Pi network configuration and target IP identification.

Once the target IP address was identified, the Streamlit interface started the MedIDS system with NFStream for real-time network traffic flow extraction. Meanwhile, the Kali machine generated real-time benign traffic by sending ICMP ping packets to the Raspberry Pi device, simulating normal traffic flow within the IoMT network. NFStream captured traffic flows in real time, while the lightweight Decision Tree classified the traffic as benign. The

Streamlit interface displayed live network monitoring results without generating alerts. These results indicate that the proposed IDS effectively distinguished normal traffic from malicious activity.



Figure 58. Real-time benign traffic monitoring using MedIDS.

To test the effectiveness of MedIDS in detecting attacks, a UDP flood attack was generated from Kali using the hping3 command. UDP flood attack sends a huge number of UDP packets toward the Raspberry Pi device within a short period, and simulates denial-of-service behaviour, resulting in abnormal network traffic behaviour. NFstream captured the abnormal behaviour of network flows and forwarded them to the decision tree model to classify, and immediately, the Streamlit interface displayed real-time attack detection notification and pop-up alerts to inform the user of the network intrusions.

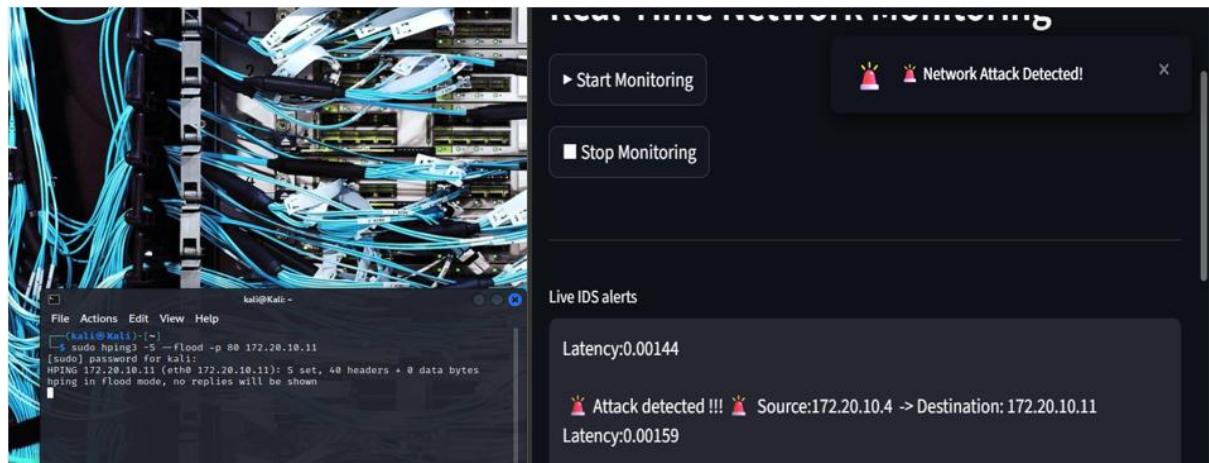


Figure 59. Real-time UDP flood attack detection and alert pop-up.

Moreover, the MedIDS system provides automatic email alerts whenever there is attack traffic. Each email alert contains a warning message showing the source and destination IP

addresses of the malicious traffic. This will enhance real-time intrusion awareness and support remote monitoring of attacks in IoMT networks.

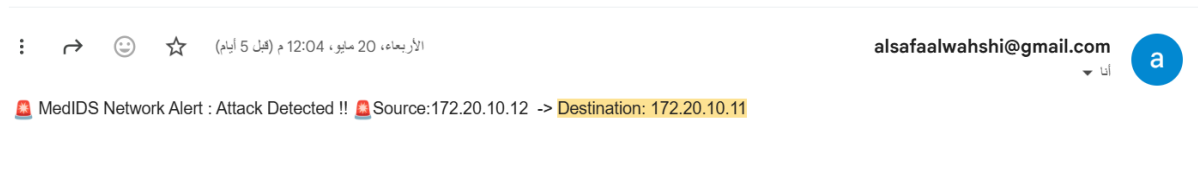


Figure 60. Email notification after network attack detection.

5.3.2 Real-Time Malware Image Detection

The real-time malware image detection experiment aimed to test the effectiveness of the proposed IDS in classifying malware and benign image samples in real-time. A sample image is uploaded through Streamlit, MobileNet processes the image, while TensorFlow Lite inference is executed on the Raspberry Pi.

When uploading a benign image sample through the interface, the TensorFlow Lite MobileNetV2 model processed the image and displayed the correct classification result, which is benign, without generating alerts. This demonstrates the ability of the TensorFlow Lite MobileNetV2 model for real-time image inference within the Raspberry Pi Zero 2W edge device.

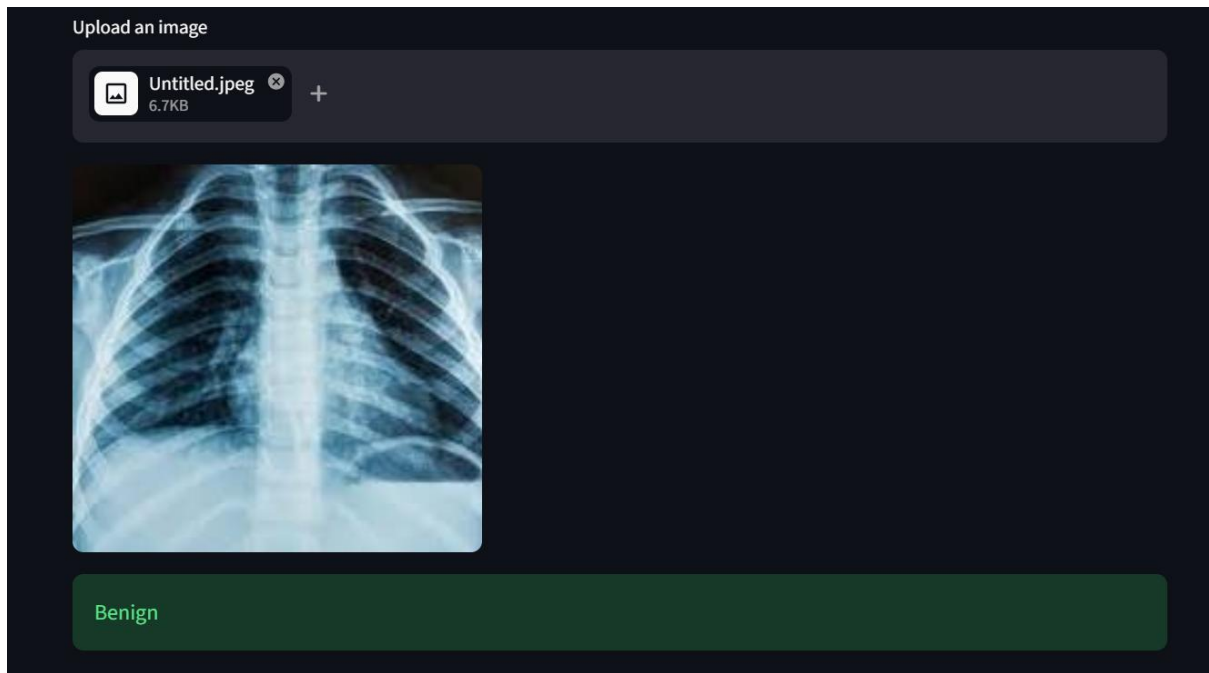


Figure 61. Benign image classification.

Another case is classifying malware image samples. When uploading a malware sample through the Streamlit interface, the model analyzed the sample and successfully classified it as malware. Then immediately after detection, a real-time malware detection notifications and pop-up alerts are displayed on the interface to inform the user about the detected malware image.

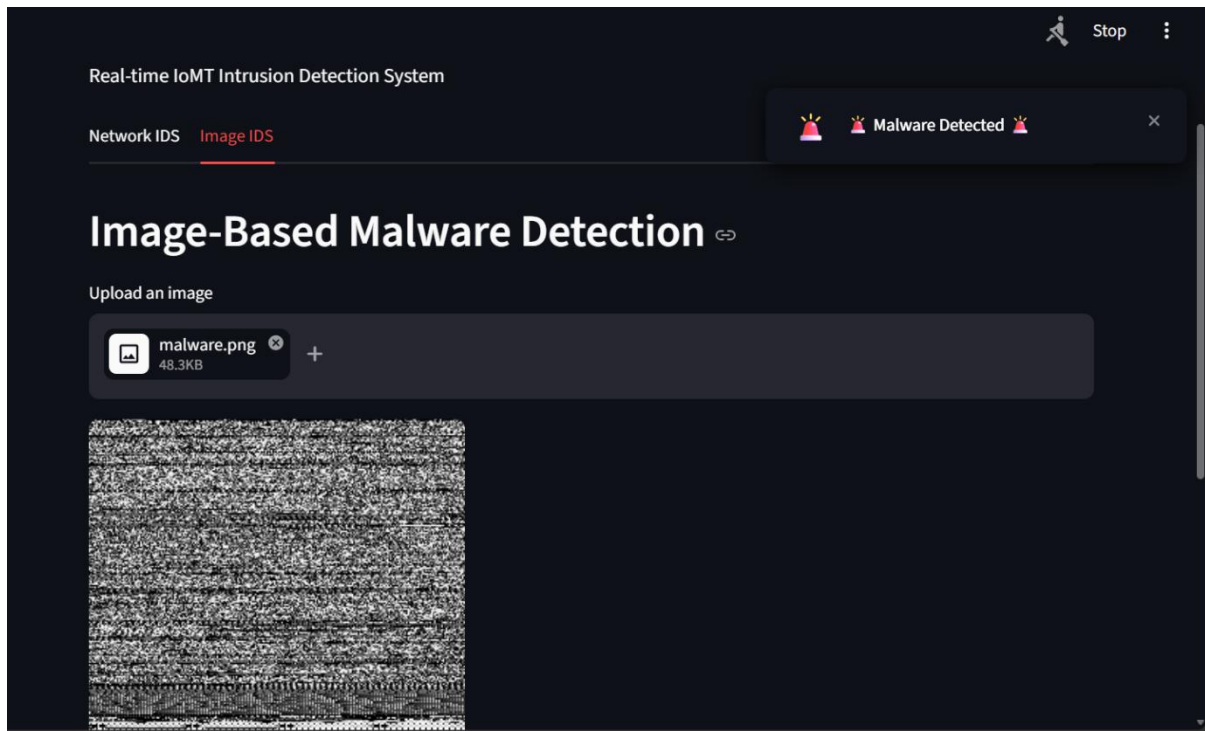


Figure 62. Malware image classification.

Moreover, the MedIDS system automatically send email notification when a malware image sample is detected. The email contains a warning message stating a malware image had been identified by MedIDS. This improves real-time malware awareness and supports remote monitoring within IoMT healthcare environment.

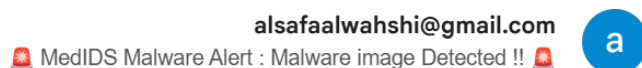


Figure 63. Email notification after malware image detection.

The real-time attack simulations showed that MedIDS works well on the Raspberry Pi Zero 2W. The network-based IDS distinguishes between benign and attack traffic and successfully generates fast attack alerts. The image-based IDS was able to classify image samples and provide real-time alerts. Integrating Streamlit alerts and email notifications improved intrusion awareness, confirming that MedIDS is suitable for lightweight IoMT edge deployment.

Chapter 6. Conclusion

This chapter presents the conclusion of the MedIDS research, summarizes key findings, discusses limitations, and provides recommendations for future improvements.

6.1 Conclusion

This research paper introduced MedIDS, a lightweight hybrid Intrusion Detection System for IoMT environments. MedIDS combines two intrusion detection methods: a network-based intrusion detection using Machine Learning, and an image-based malware detection using Deep Learning. The system runs on the Raspberry Pi Zero 2W and provides real-time intrusion detection and alerts while minimizing resource usage.

To achieve the goal, two datasets were used: CICIoMT2024 for network intrusion detection and MaleX for classifying malware images. Various preprocessing techniques were applied, such as dataset balancing, label encoding, feature selection, feature normalization, image resizing, and image normalization. These preprocessing steps improved data quality and ensured compatibility with the chosen machine learning and deep learning models.

For the network-based intrusion detection models, Decision Tree, Logistic Regression, and Naïve Bayes were evaluated. Experiments showed that the Decision Tree achieved the best performance with the highest accuracy, precision, recall, and F1-score while maintaining low computational requirements. As a result, the Decision Tree model was selected for deployment in the MedIDS framework.

For the image-based intrusion detection, the MobileNetV2 model was selected because of its lightweight nature and suitability for edge devices. Using the MaleX dataset, transfer learning, and fine-tuning, the model achieved good performance while maintaining both fast speeds and low resource utilization. This shows MobileNetV2 effectiveness for malware detection tasks.

The two models were integrated into one Streamlit interface, providing real-time network monitoring, malware image classification, visual alerts, and email notifications. The deployment results showed that both models can successfully operate on the Raspberry Pi Zero 2W edge device without requiring cloud-based processing. Measured CPU utilization, memory

consumption, and detection latency show that the proposed framework is suitable for resource-constrained IoMT environments and supports real-time intrusion detection.

Overall, the findings indicate that lightweight Machine Learning and Deep Learning models can effectively secure IoMT environments while staying compatible with low-resource edge devices. By integrating network traffic analysis and malware image classification, MedIDS overcomes several limitations of existing IDS solutions. It supports the development of practical cybersecurity measures for modern healthcare settings.

6.2 Limitations

Despite the promising results achieved by the proposed MedIDS framework, several limitations remain.

First, the network-based IDS was evaluated using binary classification, grouping all attack categories into a single class. As a result, the system detects malicious activity but does not differentiate between different attack types.

Second, Real-time attack validation was limited to UDP Flood attacks. Expanding real-time testing to include other attack types would provide a more comprehensive evaluation of the framework.

Third, the MaleX dataset was sampled and reduced to a small subset for model training due to computational and time constraints. While this subset demonstrated strong classification performance, incorporating a larger portion of the dataset could potentially enhance the model's generalization.

Fourth, the MaleX dataset primarily contains Windows executable malware and may not comprehensively represent threats targeting Internet of Medical Things (IoMT) and embedded healthcare devices.

Finally, for the deployment, Raspberry Pi Zero 2W was used, which has less processing power and memory than other edge computing systems. These hardware constraints limited the complexity of deployed models and guided decisions regarding dataset sampling and the selection of lightweight models.

6.3 Future Works

There are several ways to further improve the proposed MedIDS framework.

First, future research could shift the network intrusion detection system from binary to multi-class classification. This would allow it to recognize various attack types such as DDoS, DoS, spoofing, reconnaissance, and MQTT-based attacks.

Second, alternative lightweight machine learning and deep learning models should be evaluated and compared with the current approach to enhance detection accuracy and computational efficiency.

Third, the framework could support more IoMT communication protocols and larger healthcare networks, making it more adaptable for real-world use.

Finally, Future studies might test MedIDS on more powerful devices like the Raspberry Pi 4 and 5, and NVIDIA Jetson Nano. Such devices offer better processing power, more memory, and hardware acceleration, which could handle bigger datasets and more complicated deep learning models, while maintaining real-time intrusion detection performance.

In conclusion, the proposed MedIDS framework shows that lightweight hybrid intrusion detection systems work well to protect IoMT environments from cyber threats while remaining suitable for deployment on resource-constrained edge devices. This research sets a solid foundation for creating smart, lightweight, and real-time healthcare security solutions in the future.

Appendix

Appendix A – MedIDS Source Code Repository

The complete source code developed for this research is available at:

[GitHub - alsafaalwahshi-ops/MedIDS: A lightweight hybrid Machine Learning Intrusion Detection System \(IDS\) for Internet of Medical Things \(IoMT\) environments Using Network Traffic Analysis and Malware Image Classification on Raspberry Pi · GitHub](#)

References

- [1] M. Virthachalam, V. K. Stephen, and A. R. Manalang, "Analysis of IOMT in EMS the Sultanate of Oman Identification of CVD by Wearable E-Textile Sensors," in *2023 Second International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*, 2023, doi: 10.1109/ICEEICT56924.2023.10157631.
- [2] J.-P. A. Yaacoub, M. Noura, H. N. Noura, O. Salman, E. Yaacoub, R. Couturier, and A. Chehab, "Securing internet of medical things systems: Limitations, issues and recommendations," *Future Generation Computer Systems*, vol. 105, pp. 581–606, 2020, doi: 10.1016/j.future.2019.12.028.
- [3] N. Faruqui, M. A. Yousuf, M. Whaiduzzaman, A. Azad, S. A. Alyami, P. Liò, M. A. Kabir, and M. A. Moni, "SafetyMed: A Novel IoMT Intrusion Detection System Using CNN-LSTM Hybridization," *Electronics*, vol. 12, no. 17, p. 3541, Aug. 2023, doi: 10.3390/electronics12173541.
- [4] M. Akram, W. Khan, M. D. Rasheed, M. Imran, M. W. Iqbal, A. Delshadi, and M. Sultana, "Cybersecurity Risk Assessment Model for Internet of Medical Things (IoMT) Devices in Healthcare Systems," *Spectrum of Engineering Sciences*, vol. 4, no. 3, pp. 312–326, Mar. 2026, doi: 10.5281/zenodo.18951916.
- [5] A. Naghib, F. S. Gharehchopogh, and A. Zamanifar, "A comprehensive and systematic literature review on intrusion detection systems in the internet of medical things: current status, challenges, and opportunities," *Artif. Intell. Rev.*, vol. 58, no. 114, Jan. 2025, doi: 10.1007/s10462-024-11101-w.
- [6] A. Singh, R. M. Parizi, Q. Zhang, K.-K. R. Choo, and A. Dehghantanha, "Blockchain smart contracts formalization: Approaches and challenges to address vulnerabilities," *Computers & Security*, vol. 88, p. 101654, 2020.

- [7] D. Koutras, G. Stergiopoulos, T. Dasaklis, P. Kotzanikolaou, D. Glynos, and C. Douligeris, "Security in IoMT Communications: A Survey," *Sensors*, vol. 20, no. 17, p. 4828, Aug. 2020, doi: 10.3390/s20174828.
- [8] V. Simadiputra and N. Surantha, "Rasefiberry: Secure and efficient Raspberry-Pi based gateway for smarthome IoT architecture," *Bulletin of Electrical Engineering and Informatics*, vol. 10, no. 2, pp. 1035–1045, Apr. 2021, doi: 10.11591/eei.v10i2.2741.
- [9] W. Riyadi, Kurniabudi, Jasmir, Y. Novianto, D. Kisbianty, and X. Sika, "A Hybrid IG-PCA and Machine Learning Approach for Accurate Intrusion Detection in IoMT with Imbalanced Data," *Journal of Information and Organizational Sciences*, vol. 49, no. 2, pp. 345–359, 2025, doi: 10.31341/jios.49.2.11.
- [10] Y. Hosain and M. Cakmak, "XAI-XGBoost: an innovative explainable intrusion detection approach for securing internet of medical things systems," *Scientific Reports*, vol. 15, Art. no. 22278, 2025, doi: [10.1038/s41598-025-07790-0](https://doi.org/10.1038/s41598-025-07790-0).
- [11] E. Alalwany, B. Alsharif, Y. Alotaibi, A. Alfahaid, I. Mahgoub, and M. Ilyas, "Stacking Ensemble Deep Learning for Real-Time Intrusion Detection in IoMT Environments," *Sensors*, vol. 25, no. 3, p. 624, Jan. 2025, doi: 10.3390/s25030624.
- [12] W. El-Shafai, I. Almomani, and A. AlKhayer, "Visualized malware multi-classification framework using fine-tuned CNN-based transfer learning models," *Applied Sciences*, vol. 11, Art. no. 6446, 2021, doi: [10.3390/app11146446](https://doi.org/10.3390/app11146446).
- [13] D. A. Noever and S. E. Miller Noever, "Deep Learning Classification Methods Applied to Tabular Cybersecurity Benchmarks," *Int. J. Netw. Secur. Its Appl.*, vol. 13, no. 3, pp. 1–13, May 2021, doi: 10.5121/ijnsa.2021.13301.
- [14] X.-H. Nguyen, X.-D. Nguyen, H.-H. Huynh, and K.-H. Le, "Realguard: A lightweight network intrusion detection system for IoT gateways," *Sensors*, vol. 22, Art. no. 432, 2022, doi: [10.3390/s22020432](https://doi.org/10.3390/s22020432).

- [15] A. Aljuhani, A. Alamri, P. Kumar, and A. Jolfaei, "An Intelligent and Explainable SaaS-Based Intrusion Detection System for Resource-Constrained IoMT," *IEEE Internet of Things Journal*, vol. 11, no. 15, pp. 25454–25463, Aug. 1, 2024, doi: 10.1109/JIOT.2023.3327024.
- [16] M. Alazab, "IntruSafe: a FCNN-LSTM hybrid IoMT intrusion detection system for both string and 2D-spatial data using sandwich architecture," *Neural Computing and Applications*, vol. 37, pp. 23395–23422, 2025, doi: [10.1007/s00521-025-11527-5](https://doi.org/10.1007/s00521-025-11527-5).
- [17] M. Wazid, A. K. Das, J. J. P. C. Rodrigues, S. Shetty, and Y. Park, "IoMT Malware Detection Approaches: Analysis and Research Challenges," *IEEE Access*, vol. 7, pp. 182459–182476, Dec. 2019, doi: 10.1109/ACCESS.2019.2960412.
- [18] S. Dadkhah et al., "CICIoMT2024: Attack vectors in healthcare devices-A multi-protocol dataset for assessing IoMT device security," *Preprints*, 2024, doi: [10.20944/preprints202402.0898.v1](https://doi.org/10.20944/preprints202402.0898.v1).
- [19] R. Kaur, R. Kumar, S. Kaur, G. Singh, A. Kaur, and S. Singh, "Machine learning for diabetes prediction: Performance analysis using Logistic Regression, Naïve Bayes, and Decision Tree models," *Healthcraft Front.*, vol. 2, no. 4, pp. 169–187, 2024, doi: 10.56578/hf020401.
- [20] S. Riyadi, F. A. Abidin, and N. Audita, "Comparison of ResNet50V2 and MobileNetV2 models in building architectural style classification," in *Proc. 2024 Int. Conf. Intell. Syst. Comput. Vis. (ISCV)*, 2024, doi: 10.1109/ISCV60512.2024.10620099.
- [21] R. W. Anwer, M. Abrar, M. Ullah, A. Salam, and F. Ullah, "Advanced intrusion detection in the industrial Internet of Things using federated learning and LSTM models," *Ad Hoc Netw.*, vol. 178, art. no. 103991, 2025, doi: 10.1016/j.adhoc.2025.103991.
- [22] K. Rzepka, P. Szary, K. Cabaj, and W. Mazurczyk, "Performance evaluation of Raspberry Pi 4 and STM32 Nucleo boards for security-related operations in IoT environments," *Comput. Netw.*, vol. 242, art. no. 110252, 2024, doi: 10.1016/j.comnet.2024.110252.